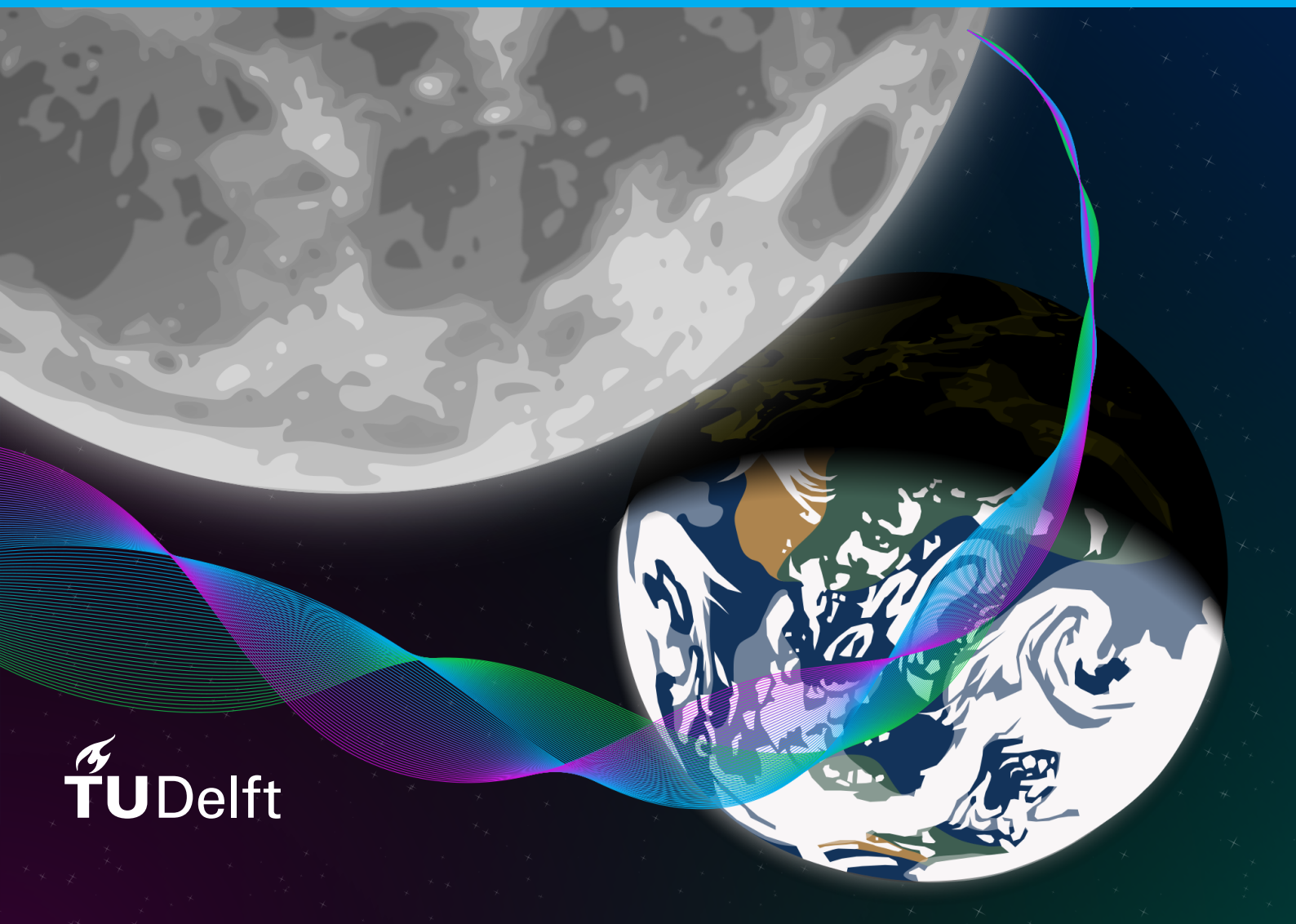
Orbital ascent trajectory computation using reinforcement learning

Preliminary thesis

A. M. de Windt



TU Delft

Orbital ascent trajectory computation using reinforcement learning

Preliminary thesis

by

A. M. de Windt

Student number: 4134249

TU Delft

List of Figures

2.1 Optimal control closed and open loop solutions. 3

3.1 Example reward distribution for a 5-armed bandit problem. 14
3.2 Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies. 15
3.3 Grid world with the safe and optimal paths learned by on-and off-policy methods, where the hashed area is the cliff. 19
3.4 Example reward distribution for a 5-armed bandit problem. 19
3.5 Artificial Neural Network[51]. 22
3.6 Minorize-Maximization algorithm 27
3.7 Example state and possible actions. 27
3.8 Learned policy at each memory state. 28
3.9 Mean rewards with varying learning rate and discount factor. 29
3.10 Simulation reference frames and states. 30
3.11 PID controller. 31
3.12 Reward history during training. 32
3.13 KL-divergence, policy and value function losses during training. 33

4.1 Histogram of the maximum norm of the position error over 1000 simulations in meters.[8] 35
4.2 Plotted here are 100 fully trained trajectories for both the V-bar and R-bar approaches [3]. 36
4.3 Sample spacecraft pose tracking and docking task. [12]. 37
4.4 Touchdown statistics [9] 37
4.5 Earth-Mars trajectory with fixed timestep [32] 38
4.6 Earth-Mars mission mass history with fixed timestep [32] 39
4.7 Earth-Mars trajectory with variable timestep [32] 40
4.8 Earth-Mars mission mass history with variable timestep [32] 40
4.9 Reference trajectories [32] 41

A.1 Powered flight trajectory[52]. 50
A.2 Possible variation of V 53
A.3 Optimal $V(t)$ for different options for Φ[52]. 54
A.4 Gravity turn[52]. 55

List of Tables

3.1 Suborbital launch vehicle properties. 30

4.1 Monte Carlo Results [32] 39

Symbols

G_t return (cumulative discounted reward) following time t.

$G_t^{\lambda a}$ λ-return, corrected by estimated action values.

$G_t^{\lambda s}$ λ-return, corrected by estimated state values.

$G_{t:h}$ flat return (uncorrected, undiscounted) from $t+1$ to h.

$G_{t:h}^{\lambda a}$ truncated, corrected λ-return, with action values.

$G_{t:h}^{\lambda s}$ truncated, corrected λ-return, with state values.

Q, Q_t array estimates of action-value function q_π or q_*.

V, V_t array estimates of state-value function v_π or v_*.

$\in$ is an element of, e.g., $s \in \mathcal{S}$.

$|\mathcal{S}|$ number of elements in $\mathcal{S}$.

$\mathcal{A}$ set of all actions.

$\mathcal{R}$ set of all possible rewards, a finite subset of $\mathbb{R}$.

$\mathcal{S}$ set of all states.

$\mathcal{S}^+$ set of all states, including the terminal state.

$\subset$ subset, e.g., $\mathcal{R} \subset \mathbb{R}$.

a an action.

$p(s', r|s, a)$ probability of transition to state s' with reward r, from state s and action a.

$p(s'|s, a)$ probability of transition to state s', from state s and action a.

$q_*(s, a)$ value taking action a in state s under the optimal policy.

$q_\pi(s, a)$ value taking action a in state s under policy π.

r a reward.

$r(s, a, s')$ expected immediate reward on transition from s to s' under action a.

s a state.

$v_*(s)$ value of state s under optimal policy.

$v_\pi(s)$ value of state s under the optimal policy π (expected return).

Greek symbols

π policy, decision-making rule.

$\pi\left(a|s,\ \boldsymbol{\theta}\right)$ probability of taking action a in state s given parameter $\boldsymbol{\theta}$.

$\pi\left(a|s\right)$ probablity of taking action a in state s under *stochastic* policy π.

$\pi\left(s\right)$ action taken in state s under *deterministic* policy π.

Contents

List of Figures iii

List of Tables v

1 Introduction 1

2 Orbital ascent trajectory optimization 3
- 2.1 Trajectory Optimization . . . 3
  - 2.1.1 Closed and open loop optimal control solutions . . . 3
  - 2.1.2 Trajectory optimization problem definition . . . 4
  - 2.1.3 Transcription . . . 4
- 2.2 Literature review . . . 6

3 Reinforcement learning 9
- 3.1 Literature review . . . 9
- 3.2 Finite Markov Decision Process . . . 11
- 3.3 Elements of Reinforcement Learning . . . 12
- 3.4 Expected return . . . 12
- 3.5 Policy and value functions . . . 13
- 3.6 ϵ–greedy policies . . . 13
- 3.7 Dynamic Programming . . . 14
  - 3.7.1 Policy evaluation . . . 15
  - 3.7.2 Policy improvement . . . 16
  - 3.7.3 Policy iteration . . . 17
  - 3.7.4 Generalized Policy Iteration . . . 17
- 3.8 Monte Carlo Methods . . . 17
- 3.9 Temporal Difference . . . 18
  - 3.9.1 TD Prediction . . . 18
  - 3.9.2 Sarsa: On-policy TD Control . . . 18
  - 3.9.3 Q-learning: Off-policy TD control . . . 18
  - 3.9.4 n-step Bootstrapping . . . 19
- 3.10 Approximate solutions . . . 20
  - 3.10.1 Stochastic Gradient Decent methods . . . 20
  - 3.10.2 Adam . . . 21
  - 3.10.3 Artificial Neural Networks . . . 22
  - 3.10.4 LSTM Neural Networks . . . 23
- 3.11 Policy Gradient Methods . . . 23
  - 3.11.1 Policy parameterization . . . 23
  - 3.11.2 Policy gradient theorem . . . 24
  - 3.11.3 REINFORCE . . . 24
  - 3.11.4 REINFORCE with baseline . . . 25
  - 3.11.5 Actor-Critic . . . 26
  - 3.11.6 PPO . . . 26
- 3.12 Example: Quadrotor guidence law . . . 27
- 3.13 Example: Suborbital flight . . . 28
  - 3.13.1 Simulator . . . 28
  - 3.13.2 The environment . . . 30
  - 3.13.3 The agent . . . 31
  - 3.13.4 PID controller . . . 31
  - 3.13.5 Results . . . 31

4 State of the art **35**
4.1 Asteroid hovering . . . 35
4.2 Proximity operation guidance . . . 36
4.3 Precision planetary powered landing . . . 37
4.4 Low-thrust spacecraft orbital change manoeuvre . . . 37

5 Conclusion **43**

A Full analytical solution to orbital ascent trajectory optimization **47**
A.1 Single stage performance . . . 47
A.2 Optimization of multistage rockets . . . 49
A.3 Optimal thrust attitude angle policy . . . 50
A.4 Optimal propellant utilization policy . . . 52
A.5 Gravity turn . . . 55

Bibliography **57**

1 Introduction

Thanks to advancements in technology and an increasingly widespread use of satellite services, demand for space access has been steadily rising the past decades. This growth in the satellite launch services market has lead to the development of new launchers by both government agencies and an emerging private sector.

One often overlooked, but crucial step during the design process of any product is the performance evaluation of proposed designs. This is the process of determining a products ability to meet the design requirements. Too low of a performance, and the product will not be able to fullfil its purpose. To high of a performance and the product may be over engineered or inefficient.

In the case of a launcher the performance may be measured by how much payload mass it can bring to it's target range of orbits. In other words, in order to maximize the performance of a launcher, the propellant mass fraction of the upper stage needs to be reduced.

A successful orbital insertion requires the launcher to carefully follow an optimized trajectory. This trajectory has a massive effect on the maximum performance of the launcher, but unfortunately finding the optimal trajectory is not a trivial. This problem falls under what is known as optimal control, a class of problems where the goal is to design a controller that optimizes a performance measure of a dynamical system's behavior over time, [2, 51].

A commonly used class of methods to compute orbital ascent trajectories is trajectory optimization. This is a set of mathematical techniques used to find the desired behavior of a dynamic system by finding the inputs as a function of time, [2, 16]. Trajectory optimization can be split in two groups, indirect and direct methods.

Indirect methods are more commonly used when computing orbital ascent trajectories, [53], due to their higher accuracy, but some direct methods have also been proposed, [5, 14, 45]. More recently heuristic optimization methods, such as Participle Swarm Optimization (PSO) and Genetic Algorithm (GA), have been proposed in order to circumvent the initialization issues associated with indirect methods, [41, 59].

Using any of these methods to compute orbital ascent trajectories is computationally intensive and only yields open loop solutions for one particular trajectory. This is not an issue when calculating the trajectory for a launch during normal operations, but it is during the design process of a launch vehicle.

A fast method to get a large number of trajectories is desired during the design process in order to, for example perform a sensitivity analyses on the design parameters, incorporate the trajectory solver into a larger Multidisciplinary Design Optimization (MDO) project, or continuously evaluate and monitor the feasibility of a changing design.

This thesis proposes the use of Reinforcement Learning (RL) as a method to compute closed-loop solutions for orbital ascent trajectory problems. With enough training an RL agent is able to learn to solve complex and non-linear problems. Thus the goal is to develop an RL agent that is able to

compute optimal trajectories for a wide range of initial states and launcher configurations within a single simulation run.

Even if generating optimal trajectories using RL ends up being unfeasible, it may still be able generate close to optimal trajectories, which may either be acceptable during the preliminary design phase, as a component of an MDO project, or serve as an initial guess for a more refined optimization algorithm.

Thus the main research objective and questions for this thesis are:

"To develop a tool capable of computing large batches of orbital ascent trajectories for a wide range of initial conditions and launcher configurations, by taking advantage of Reinforcement Learning and investigate its effectiveness with respect to existing methods, using metrics such as the accuracy of the results and computational requirements."

How are orbital ascent trajectories currently computed?

- What are the challenges faced when computing an orbital ascent trajectory?
- What methods are currently used or being researched to solve this problem?
- What are the limitations of methods?

What is Reinforcement Learning?

- How does RL work?
- What is RL capable of?
- How has RL been used so far?
- What state of the art technological developments make use of RL?

How can Reinforcement Learning methods be used to solve the orbital ascent trajectory optimization problem?

- What advantages do RL methods have over existing methods?
- What properties of RL can be used to circumvent the limitations of current orbital ascent trajectory optimization methods?
- Are RL based orbital ascent trajectory computation methods capable of achieving the accuracy of existing methods?
- How many times should a trained agent be used to make the use of RL cost effective.
- May these RL methods be used to compliment or improve the performance of existing methods?

Chapter 2 starts with the theory of behind trajectory optimization and is directly followed by a review of the state of the art methods used in orbital ascent trajectory optimization.

Chapter 3 starts with a history on RL and continuous with a literature review of latest developments in RL. This is then followed by several sections explaining the theory behind RL ending with the more advanced methods that will be used throughout the main thesis project. The chapter then concludes with two examples demonstrating the different methods.

The next chapter contains a review of the state of the art research on RL in spacecraft guidance, and finally the report is concluded with an overview of research questions and the proposed experiment to be conducted during the main thesis.

The use of RL in spacecraft guidance, navigation and control opens up the possibilities to solve future problems as space exploration becomes more prevalent. For example thanks to its abilities to generalize and adapt in real time, it may be used in situations that are difficult to forsee or too cumbersome to manually take into account, such as hovering and manoeuvering around the non-uniform, rotating gravitational field of an asteroid, [8, 56], or performing docking operations with non-cooperative target objects in cluttered environments, [3, 12, 13].

2

Orbital ascent trajectory optimization

The problem of placing a satellite in orbit can be solved by achieving a required velocity at a certain position in space. Unfortunately reaching this goal is not a trivial problem. In order for a launcher to carry out it's mission as efficient as possible it must follow a carefully optimized trajectory. Failure to do so results in wasted potential performance.

This chapter starts with a background on trajectory optimization, then continues with a literature review of state of the art methods, and ends with a simplified analytical solution.

2.1. Trajectory Optimization

Unfortunately due to the complex non-linear dynamics and constraints, and high dimensionality of the problem it is not possible to calculate the optimal orbital ascent trajectory analytically and thus numerical solutions are necessary. This has been a problem since the beginning of space exploration and thus countless years of research have resulted in the use and further development of *trajectory optimization*, a set of mathematical techniques used to find the ideal or desired behavior of a dynamical system by finding the inputs, referred to as *controls*, to the system as functions of time.

2.1.1. Closed and open loop optimal control solutions

In itself trajectory optimization is class of methods used to compute solutions for optimal control problems. Within optimal control there are two types of solutions, closed-loop and open-loop.

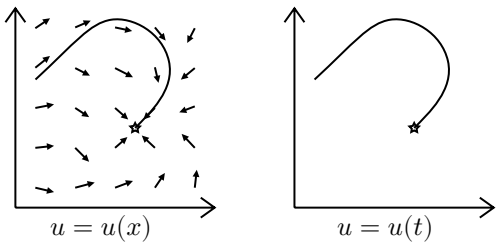


Figure 2.1: Optimal control closed and open loop solutions.

Closed-loop solutions provide the optimal controls for the entire state space, $u(x)$, thus advantages of these solutions include the possibility to compute the global optimal solution and the ability to apply the solution directly into real systems. The drawback is that these solutions are difficult to compute, specially when applied to higher dimensional problems. Dynamic Programming and the main topic of thesis, Reinforcement Learning work by finding an optimal policy. This is a mapping of states to actions and thus provide closed-loop solutions to optimal control problems [16]. Chapter 3 continues further with these methods.

On the other hand, trajectory optimization computes open-loop solutions consistent of an initial state

and a sequence of controls defined as a function of time, $u(t)$. Together these define a single trajectory the system takes through the state space. The advantage of these solutions is that they are easier to compute, and thus more practical for higher dimensional problems.

The disadvantage is that since these solutions are based on computations in the local state space they do not necessarily converge to the global optimal and are sensitive to the initial guess. Another disadvantage is that these solutions require a stabilizing controller in order to keep the system on the predefined trajectory when applied to real systems [16].

2.1.2. Trajectory optimization problem definition

There are many ways to define trajectory optimization problems [43], but assuming the problem consists of a single-phase and is continues in time, the problem can be described with an objective function and a set of constraints as shown is Equation 2.1 [16].

$$
\begin{aligned}
\underset{t_0, t_F, x(t), u(t)}{\text{minimize}} \quad & J\left(t_0, t_F, x(t_0), x(t_F)\right) + \int_{t_0}^{t_F} w(\tau, x(\tau), u(\tau)) d\tau \\
\text{subject to} \quad & \dot{x}(t) = f(t, x(t), u(t)) \qquad \text{(system dynamics)} \\
& h(t, x(t), u(t)) \leq 0 \qquad \text{(non-linear path constraints)} \\
& g(t_0, t_F, x(t_0), x(t_F)) \leq 0 \qquad \text{(non-linear boundary constraints)} \\
& x_{low} \leq x(t) \leq x_{upp} \qquad \text{(path bounds on state)} \\
& u_{low} \leq u(t) \leq u_{upp} \qquad \text{(path bounds on control)} \\
& t_{low} \leq t_0 < t_F \leq t_{upp} \qquad \text{(bounds on initial and final time)} \\
& x_{0,low} \leq x(t_0) \leq x_{0,upp} \qquad \text{(bounds on initial state)} \\
& x_{F,low} \leq x(t_F) \leq x_{F,upp} \qquad \text{(bounds on final state)}
\end{aligned}
\tag{2.1}
$$

The objective function consists of two terms, a boundary objective, $J(\cdot)$, also known as the *Mayer term*, and a path integral along the entire trajectory with integrant, $w(\cdot)$, also known as the *Lagrange term*. Such an objective function is said to be in *Bolza form*.

The system dynamics constraint is typically non-linear and describes how the system changes with time. The non-linear path constraints enforce restrictions along the path, for example airspace restrictions. Non-linear boundary constraints restrict the initial and final state of the system. The path bounds put restrictions on the states and controls, for example maximum actuator settings or aerodynamic loads. The bounds on initial and final state and time can be used to for example, set the initial launch location and final position and velocity needed for the target orbit.

2.1.3. Transcription

The goal of trajectory optimization is to minimize an objective function in the form of $J(f(t))$, where the decision variable $f(t)$ is an arbitrary vector function. In contrast, *parameter optimization* minimizes objective functions in the form of $J(x)$, where the decision variables in vector x are a finite set of real numbers. This makes trajectory optimization a more difficult problem since having a function as decision variable is the equivalent of having an infinite set of real numbered decision variables [16].

Transcription is the process of converting a trajectory optimization problem into a constrained parameter optimization problem. Thus the goal of transcription is to convert a infinite-dimensional problem into a finite-dimensional approximation, which can then be solved using parameter optimization methods [2]. The three fundamental steps of the transcription method are:

1. Convert the dynamic system into a problem with a finite set of variables.
2. Solve the finite-dimensional problem using a parameter optimization method.
3. Asses the accuracy of the finite-dimensional problem approximation and if necessary repeat the transcription and optimization steps.

Direct and indirect methods
Transcription methods can be split into two categories, direct and indirect methods.

Indirect methods minimize the objective function by deriving the necessary conditions for optimal control as found in *J. Betts* [2] and then defining a new multi-point boundary value problem. The new problem is then solved using *calculus of variations* by analytically deriving the first and second variations of the new objective function with respect to the controls and using these to find the minimum with numerical root finding methods [2, 16, 53].

In contrast direct methods work by constructing a sequence of points, $x_0, x_1, ..., x^*$, such that the objective function is minimized with each subsequent point. Unlike indirect methods, direct methods only need to compare values for the objective function.

The advantage of indirect methods is the higher accuracy and a better measure for the accuracy of the solution. However these methods require prior knowledge of the system and even if this knowledge is available the required analytical derivations may be too difficult or not possible.

Furthermore indirect methods are not robust. The region of convergence tends to be smaller, thus better initialization is required. Initializing indirect methods require the initialization of the *adjoint variables*, which since they have no physical representation, are more difficult to initialize [2, 16].

Direct shooting
In direct single shooting methods the trajectory is approximated using a simulation. The decision variables in the parameter optimization problem are the initial state and an open-loop parametrization of the controls along the trajectory.

An extension to direct single shooting methods are direct multiple shooting methods. Instead of representing the entire trajectory as a single simulation, the trajectory is segmented and each segment is represented by a simulation. Multiple shooting tends to be more robust than single shooting [16].

Direct collocation
Direct collocation methods work by approximating all of the continues functions as polynomial splines. As an example take the following problem, compute the trajectory to move a block between two points. Where x is the position, v the velocity and u the control. The objective function minimizes the control effort by integrating the square the control.

$$\begin{aligned} \underset{x(t),v(t),u(t)}{\text{minimize}} \quad & \int_0^1 u^2(\tau)\, d\tau \\ \text{subject to} \quad & \dot{x}(t) = v(t) \\ & \dot{v}(t) = u(t) \\ & x(0) = 0 \\ & x(1) = 1 \\ & v(0) = 0 \\ & v(1) = 0 \end{aligned} \tag{2.2}$$

The first step is to discretize the trajectory by representing the continues functions by their values at specific points in time, these are known as *collocation points*.

$$\begin{aligned} t &\rightarrow t_0 \dots t_k \dots t_N \\ x &\rightarrow x_0 \dots x_k \dots x_N \\ v &\rightarrow v_0 \dots v_k \dots v_N \end{aligned}$$

The next step is to convert the continuos system dynamics to a set of constraints. This is done by equating the change in states between two collocation points to an estimate of the system dynamics

integral during the same time period. Here trapezoidal collocation is used, where the system dynamics is approximated by piece-wise linear functions. Another alternative is the Hermite–Simpson collocation method, which approximates the system dynamics using piece-wise quadratic functions.

$$x_{k+1} - x_k = \frac{1}{2}(t_{k+1} - t_k)(v_{k+1} + v_k) \tag{2.3}$$

$$v_{k+1} - v_k = \frac{1}{2}(t_{k+1} - t_k)(u_{k+1} + u_k) \tag{2.4}$$

$$\tag{2.5}$$

These constraints are known as *collocation constraints* and are applied to each pair of collocation points. Finally the objective function can then be approximated using quadrature by converting it to a summation of piece-wise functions between each collocation point. Using trapezoidal quadrature yields,

$$\underset{u(t)}{\text{minimize}} \int_0^1 u^2(\tau)\, d\tau \quad \approx \quad \underset{u_0 \dots u_N}{\text{minimize}} \sum_{k=0}^{N-1} \frac{1}{2}(t_{k+1} - t_k)(u_{k+1}^2 + u_k^2) \tag{2.6}$$

As result all continues functions have been replaced by constraints and the decision variables $x(t)$, $v(t)$ and $u(t)$ have been replaced by the finite sets of decision variables, $x_0 \dots x_N$, $v_0 \dots v_N$ and $u_0 \dots u_N$.

Indirect shooting and collocation
Shooting and collocation methods for discretization can be used by indirect methods as well by applying them to the boundary value problem.

2.2. Literature review

Despite its disadvantages, indirect methods are widely used when computing space flight trajectories. The main reason is the low accuracy associated with direct methods. In practice when using direct methods the minimum functional value found can be expected to have an error of about one percent which can be a crucial part of the payload in a space flight mission. Increasing the finite-dimensional space doesn't necessarily yield better results [53].

One such indirect method is the Sequential Gradient-Restoration Algorithm (SGRA). Developed during the years 1968-86 by Miele *et al.* [28], this method has been successfully applied to atmospheric flight [29], space flight [23, 25] and flights at the interface between atmosphere and space [21, 22, 24, 30]. Applications and extensions have been developed all around the world with a notable example being the implementation of SGRA used at the NASA Johnson Space Center under the code name SeGRAm [31, 44].

SGRA is a sequential algorithm that alternates between gradient and restoration phases. In the gradient phase variations on the decision variables are computed that reduce the value of the augmented objective function, while avoiding excessive constraint violations. Then in the restoration phase variations on the decision variables are computed that reduce the constraint errors while avoiding excessive change in the decision variables.

In 2003 Miele *et al.* developed an extension to SGRA called Multiple-Subarc Gradient-Restoration Algorithm (MSGRA). The reason was to improve the robustness of the method and enlarge the field of applications. MSGRA can be applied to optimal control problems involving multiple subsystems as well as discontinuities in the state and control variables at the interface between contiguous subsystems [26]. MSGRA has demonstrated in being an effective tool for optimizing the trajectories of multiple stage launch vehicles [27].

A mayor disadvantage of indirect methods is the high sensitivity and difficulty to select an appropriate initial guess. In order to lessen this problem some applications apply the use of the homotopy method. The principle of the homotopy method is that a given problem is embedded into a family of problems

parametrized by a homotopic parameter. The solution to the original problem can then be found by starting with a simpler problem and gradually adjust the homotopic parameter such that in the end a solution to the original problem is found [37]. However the issue with the homotopy method is the added complexity to the solution process.

As an example Calise *et al.* [4] and, Gath and Calise [7] propose the use of a hybrid analytical/numerical approach which uses a homotopy method that starts with a nearly analytic vacuum solution that gradually introduces atmospheric effects until a converged solution is obtained. This method has the potential to be used online, since once converged, the trajectory can be updated within milliseconds using input from the guidance system due to the nearly analytic nature of the solution process [7].

Direct methods have also been proposed, such as those by Roh and Kim [45] and Jamilnia and Naghash [14], both of whom solve the problem by using direct collocation to transcribe the trajectory optimization problem into Non-Linear Programming (NLP) problems. Another direct method proposed by Coverstone-Carrol and Williams [5] uses differential inclusion concepts to remove explicit control dependence from the problem statement thereby reducing the number unknown parameters.

Recently newer techniques make use of heuristic optimization methods. A problem inherent to the methods already discussed is that since these methods are based on local computations, they are sensitive the starting guess and do not always converge towards the global optimum. On the other hand heuristic methods are global techniques which can look for the global optimum within a larger portion of the state space. The main idea behind heuristic optimization methods is that the search is done in a stochastic manner instead of a deterministic manner [43]. However the main disadvantage of heuristic methods is the lack of any convergence proof and the capability of yielding only near optimal solutions [40].

These disadvantages can be circumvented by combining indirect and heuristic methods. As an example Pontani *et al.* [38–41] propose a method for computing the optimal exoatmospheric trajectory using Participle Swarm Optimization (PSO). The proposed method starts by analytically deriving the necessary conditions for optimality just as with indirect methods, then PSO is used to numerically determine the state, control and adjoint variables. The use of a heuristic method significantly lessens the initialization issues while retaining the accuracy and measure of accuracy that come with indirect methods, without the added complexity associated with the use of homotopic methods.

A more direct heuristic optimization approach is that taken by Zheng *et al.* [59]. A pitch law is defined consisting of five successive sections, each for a single phase of flight with the first one starting at take-off and the last one ending at orbit insertion. A function with unknown parameters is chosen for each section of the pitch law, and then the unknowns are found using Genetic Algorithm (GA). Since these functions can be selected based on prior knowledge, it's possible to significantly reduce the number of unknowns. In this case there where only nine.

This is a similar approach as to that taken by Pontani and Teofilatto [41] to compute the endoatmospheric flight trajectory using orbital ascent. However in this case, the parameters where found using gradient based optimization methods instead of heuristic methods.

3

Reinforcement learning

So far the methods discussed for solving the orbital ascent optimal control problem only yield open-loop solutions. These are solutions for a single trajectory defined by an initial state and the controls as a function of time. In contrast methods such as Dynamic Programming (DP) and Reinforcement Learning (RL) produce closed-loop solutions consistent of mappings known as policies that map the controls, also known as actions, with the set of possible states instead of time.

DP was originally developed in the 1950's as a set of methods used to solve optimal control problems and are widely considered the only feasible way to solve a class of discrete stochastic optimal control problems known as Markov Decision Processs (MDPs). However DP suffers from what its author called "*the curse of dimensionality*", meaning the computational requirements grow exponentially with the number of state variables. The main reasons are because DP methods require tables large enough to hold values for the full state set and also require repeated sweeps of the entire state set.

RL extends DP by adding the ability to learn directly from raw experience. Learners use this ability to learn policies by trying out different actions and discovering the most valuable. It does this by not only looking at the reward it received by its latest action, but it also takes into account any future rewards it could receive by taking that action. This learning method coupled with the use of function approximation methods such as Artificial Neural Networks (ANNs) allows learners to learn useful policies without visiting the entire state set by generalizing between visited states and actions.

This opens up the possibility to compute closed-loop solutions for the high-dimensional orbital ascent trajectory problem that are able to represent a range of optimal or close to optimal trajectories for a wide range of initial states.

This chapter starts with the basic theories behind Reinforcement Learning, followed by a description of the more advanced methods to be used during this thesis and ends with a few examples.

3.1. Literature review

The simplest forms of Reinforcement Learning learn using what are known as a value functions that represents the utility of each state or state-action pair. Q-learning is one of the most commonly used of these methods. It has shown to always converge to the optimal policy as long as all states and actions are sufficiently sampled. [54]

Function approximation may be used to generalize between visited states and actions. In this case a function approximator is used to store and retrieve value function estimates. It offers a way to handle continuous state spaces, mitigate against state-action space explosion and generalize prior experience to previously unseen state-action pairs.

Deep Reinforcement Learning (DRL) is a class of RL methods where a Deep Neural Network (DNN) is used as the function approximator. As an example Deep Q-Networks (DQN) extend Q-learning by

using deep learning neural networks to approximate the state-action value function. This allows agents to learn policies that can reach human-level control for high-dimensional state spaces [33].

An alternative to value function based methods are policy based methods. The aim is to estimate the optimal policy directly and the value is secondary if it's calculated at all. Policy gradient methods use gradient decent to estimate the policy parameters such that the expected reward is maximized. Policy based methods allow RL to be used in the continuous action spaces. An example of such a method is Deterministic Policy Gradient (DPG) [49].

Actor-critic methods are hybrid methods that combine the benefits of policy-based and value-based algorithms. In actor-critic methods the actor represents the learned policy and the critic the learned value function when its used for bootstrapping. That is the process of updating the policy and value using the estimated values of future states[51].

Another way to use the value function in policy-based methods is to use it as a baseline when compting the parameter updates. These are known as advantage based methods. Deep Deterministic Policy Gradient (DDPG) is an extension of DPG [20] implementing both actor-critic methods and the advantage.

An intuitive way to see the advantage is that it allows agents to not only know how good the actions were, but also how much better they where than expected. Leading to reduced variance and more stable training.

One of the key issues with RL is sample efficiency. Animals are usually able to learn new tasks with just a few attempts by using prior knowledge. RL methods require too many samples before a reasonable policy is learned. This is especially an issue when collecting experience is expensive or dangerous and the problem is made worse when rewards are sparse or delayed. However several methods have been developed to mitigate these issues.

Experience replay allows agents to remember and reuse experiences that where observed in the past, thus increases the sample efficiency. In it's simplest form experiences are replayed in the same frequency in which they ocurred regardless of the significance. An alternative is to separate the experiences into buckets of good and bad experiences and select a fixed fraction from each bucket [35]. Another option which has shown to be quite effective is to prioritize experiences with important transitions based on the TD error [46].

Methods using experience replay such as DQN and DDPG have been shown to be quite effective but require large amounts of memory to store the experienced samples. In Asynchronous Advantage Actor Critic (A3C) [34] multiple instances of agents and environments are executed in parallel. Once an agent is ready it communicates back the computed gradients of the policy and value function parameters to a central parameter server and updates itself with the latest parameter values. This process is repeated periodically until the optimal policy is found. Unlike methods using experience replay which are required to use off-policy algorithms, A3C can be applied to both on and off policy algorithms.

Advantage Actor Critic (A2C) is the synchronous version of A3C. The difference is that in A2C the central parameter server waits for all agents to finish before performing the update. A2C and A3C have comparable performance [34].

Trust Region Policy Optimization (TRPO) works by avoiding the new policy from deviating too much from the previous one, thus preventing a bad update. This is done by defining a new surrogate objective function that limits the change in policy by constraining the KL-divergence between the two policies. This method results in monotonic improvements in the policy performance, and allows for multiple updates on the policy to performed per experience sample [47].

Proximal Policy Optimization (PPO) works similarly to TRPO, in that it also avoids large deviation in the policy during updates. But instead of applying a hard constraint based on the KL-divergence, the surrogate objective function uses a clipping method to apply a penalty when the two policies diverge to much [48].

In order to deal with sparse rewards *reward shaping* allows agents to learn intermediate goals leading towards the main goal by adjusting the reward function using prior knowledge such that the agents receive rewards more frequently[55]. The disadvantages of this method is that it requires prior knowledge

of the environment and the optimal policy for the new adjusted reward function is not necessarily the same as the optimal policy for the original reward function. As an example Randlov *et al.* [42] where experimenting with a bicycle agent who would turn in circles to stay upright instead of reaching its goal.

Difference (D) [57] rewards and potential-based reward shaping (PBRS) [36], are commonly used shaping approaches which have been successfully applied to a wide range of domains.

Another method for handling sparse rewards is Learning from Demonstrations (LfD). In LfD, an agent learns to perform a new task from demonstrations performed by an expert without feedback from the rewards. Obtaining sufficient high quality demonstrations may not be feasible in many cases, thus leading to suboptimal policies. However LfD is still a useful tool that can be used to initialize the agent with a good or a safe policy and further learning can then search for better policies.

For example AlphaGo [50] initializes the policy network using supervised learning on state-action pairs from recorded human experts. Then using self-play AlphaGo was able to discover new plays and learn from its mistakes to eventually achieve super human performance.

Another approach is that taken by Hester *et al.* [10] to develop Deep Q-learning from Demonstrations (DQfD). In this case the agent is initialized by placing a small set of demonstrations in the replay buffer. DQfD pre-trains solely on the demonstration data using a combination of both TD and supervised losses and eventually the agent starts learning using a combination of both demonstrated and self-generated data. This resulted in quite a capable method that outperformed standard reinforcement learning and three related methods for incorporating demonstration data into DQN.

Behavior Cloning (BC) is a class supervised learning methods that maps states to actions based on demonstrations from an expert. On the other hand Inverse Reinforcement Learning (IRL) estimates the underlying value function that justifies the actions taken by the expert. This is considered to be more robust.

A drawback of IRL algorithms is that they can be computationally expensive since they require a reinforcement learning inner loop between the cost estimation, policy training and evaluation.

An alternative to IRL inspired by Generative Adversarial Nets (GAN) is Generative Adversarial Imitation Learning. It avoids the use of the inner loop by training a policy until it is able to fool a discriminator, resulting in policies that travel the same MDP paths as the expert.

3.2. Finite Markov Decision Process

A Markov Decision Process (MDP) is a stochastic process that provides a mathematical framework used to describe problems where a learner learns from its actions to achieve an end goal. This framework is used in many areas, such as robotics, economics, manufacturing and in the case of this thesis, Reinforcement Learning.

The two main components of a MDP are the agent and the environment. The agent makes decisions, interacts with its environment and learns. The environment consists of everything else. The two are constantly interacting with each other, the agent will perform an action which changes the state of the environment and the environment responds by giving it a reward. The goal of the agent is to maximize the reward it receives from the environment.

This happens in a sequence of discrete time steps. At the beginning of each time step the environment is at a certain state s. The agent then decides based on its policy π which action a to take, this in turn changes the environment to the new state s' and the environment then gives the agent a certain reward r. In order for it to be a Markov Decision Process (MDP) this transition and the reward must satisfy the Marcov property, which means it needs to be memoryless and thus the environment's behavior must be independent of its past states. In other words, the probability of the environment to go from state s to s' and the reward r it gives back to the agent, must only be dependent on the initial state s and action a.

3.3. Elements of Reinforcement Learning

The main goal when solving an MDP problem is to find a policy π that defines the action $\pi(s)$ the agent will take in order to maximize the sum of rewards it receives. It characterizes the way in which the agent will behave at a given time by mapping the agents perceived state of the environment to the actions taken in those states. This policy may be as simple as a table or consist of several Neural Networks. The policy is the core of the agent in that it alone defines the behavior of the agent.

On each time step the environment will send the agent a reward. The goal of the agent is to maximize this reward in the long run. Thus this reward is used to tell the agent which states are good or bad. If the agent receives a low reward for a certain action it can change the policy so it takes a different action in the future.

The reward signal tells the agent what the immediate effects of its action are. However, its goal is to maximize the cumulative sum of rewards it receives over time, and thus in order for it to receive higher rewards in the future, it might be necessary for the agent to take actions that yield lower rewards with the promise that it will receive higher rewards later. So the value is an indication of the reward the agent can expect to receive in the future by taking a certain action, and thus it is the main parameter used in the decision making process.

By defining the state and action taken during step t as S_t and A_t respectively, and its corresponding reward as R_t we can define the true value of an arbitrary action a as:

$$q_*(s,a) \doteq \mathbb{E}\left[R_t \mid S_t = s, A_t = a\right] \tag{3.1}$$

If $q_*(s,a)$ was known, then the solution would be trivial. Just select the action with the highest value. So it's assumed that the action values are not known, but are instead estimated from the rewards the agent receives. The estimated value of the state-action pair (s,a) at time t is known as $Q_t(s,a)$, thus $Q_t(s,a)$ needs to be as close to $q_*(s,a)$ as possible.

Finally some Reinforcement Learning (RL) agents make use of a model for planning. This allows these agents to choose actions based on possible future situations before actually being experienced.

3.4. Expected return

The goal of the agent is to maximize the reward it receives over time. In order to formalize this goal, the *expected return* G_t can be defined as a function of the sequence of rewards. In it's simplest form this can be the cumulative sum of all rewards after time step t until the final time step T.

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + ... + R_T \tag{3.2}$$

This function assumes that there are one or more terminal states where the agent-environment interaction ends and break naturally into subsequences called *episodes*. A good example are plays of a game or any type of repeating task. Even if there are different terminal states, eg. winning or losing a game, the starting state of the next episode must be independent of the previous episode. These type of tasks are called *episodic tasks*.

However not all tasks are episodic, it is possible an RL-agent needs to solve some task that can continuo without a limit. These types of tasks are called *continuing tasks*. The expected return given by Eq. 3.2 can be problematic for these types of tasks, since the final time step would be at $T = \infty$. Thus it is possible the value it's trying to maximize is infinite as well. so to solve this problem the rewards can be *discounted*. In this approach the sum of discounted rewards, also known as the *discounted expected return*, shown in Eq. 3.3 is maximized.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{3.3}$$

It is possible to unify the expected return equations for the episodic and continuing tasks into a single one. For example, in the transition diagram for a continuos task shown below the environment transitions from states s_0 to s_1 and after its next action it ends up at a special absorbing state where all actions will result in the same state without any reward. This results in reward sequence $+1, +1, +1, 0, 0, ...$ By summing the whole infinite sequence we get the same result as summing the first T rewards, where $T = 3$ in this case. This is also true with discounting.

Thus Eq. 3.4 shows the unified equation for the (discounted) expected return for both episodic and continuing tasks. If the task at hand is continuing, then $T = \infty$, while for episodic tasks T will be some finite integer. It is possible to have $\gamma = 1$ for episodic tasks, but due to the reasons mentioned before the expected return must be discounted for continuing tasks, so $\gamma = 1$ and $T = \infty$ cannot be true at the same time.

$$G_t \doteq \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \tag{3.4}$$

One important property of the expected return used in reinforcement learning is that it is recursive as demonstrated below.

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + ... \\ &= R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + ... \right) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \tag{3.5}$$

3.5. Policy and value functions

The policy is a mapping from the state of the environment to the probabilities of the agent selecting each action. If the policy at time t is π, then $\pi(a|s)$ is the probability of selecting action a if the current state is s.

The value of a state can be different depending on the policy of the RL-agent, since different policies will result in different actions and thus different states and rewards. So $v_\pi(s)$ is the expected return when starting at state s and following policy π. v_π can be defined by:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t | \, S_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s \right], \text{ for all } s \text{ in } \mathcal{S} \tag{3.6}$$

For some problems it might be more suitable to look at the value of state-action pairs. Thus similar as to the state value above, the state-action value under policy π, denoted by $q_\pi(s, a)$, is defined by:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[G_t | \, S_t = s, A_t = a \right] = \mathbb{E}_\pi \left[\sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \middle| S_t = s, A_t = a \right], \text{ for all } s \text{ in } \mathcal{S} \tag{3.7}$$

3.6. ϵ–greedy policies

The main difference between Reinforcement Learning and other types of machine learning is that RL uses training information that evaluates the actions taken by the agent instead of a set of correct actions. This means that to determine $Q_t(a)$ the agent needs to explore and try out as many actions as possible in order to find good behavior. To do this the agent can take random actions and learn from the rewards it gets back. This is called *exploration*.

However, the main goal of the agent is to maximize the rewards it receives from the environment over time. To achieve this goal the agent can choose to take greedy actions. These are the actions with the highest value in $Q_t(a)$. This process of choosing the greedy action is known as *exploitation* since the agent is exploiting its knowledge to achieve its goal.

Unfortunately exploration and exploitation conflict with each other since in the former the agent wants to try out all possible actions, but only wants to perform a specific action if it's following a greedy policy. Thus a possible compromise is to use what is known as an ϵ-greedy policy. In these policies, the agent will choose a random action ϵ percentage of the time and the greedy action the rest of the time. This allows the agent to explore and try out new actions or old actions whose value might have changed, while also trying to achieve it's goal.

Since the agent is trying out new actions while following ϵ-greedy policies, it is able to learn faster and more than if it was following a greedy policy. To demonstrate this a k-armed bandit problem is used.

K-armed bandit problems are simple learning problems where the agent is given the option to take k different action's. There is only one state and the episodes terminate after the agent has taken it's action. Furthermore the examples in this report use zero mean, unit variance normal distributions to select the true value $q_*(a)$ of the actions. The rewards given by the environment are then selected with $q_*(a)$ mean and unit variance normal distribution. Figure 3.1 shows an example reward distribution for a 5-armed bandit.

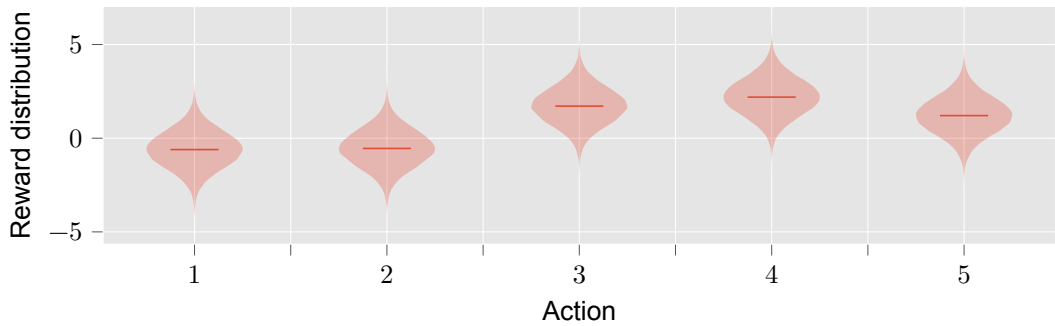


Figure 3.1: Example reward distribution for a 5-armed bandit problem.

The agents use the simple learning rule shown in Equation 3.8. Here is Q a table with the action-value function. Results for a group of 2000 10-armed bandits can be found in Figure 3.2.

$$Q(A) \leftarrow Q(A) + \alpha \left[R - Q(A)\right] \tag{3.8}$$

The average reward of all bandits are stored after each episode. Figure 3.2 shows the average reward and the percentage of agents that selected the optimal action for the greedy policy and two ϵ-greedy policies ($\epsilon = 0.01$ and $\epsilon = 0.1$).

As it can be seen when using ϵ-greedy policies the agents learn faster and are thus able to receive higher rewards sooner. On the other hand, while following the greedy policy, the agents will quickly choose the first action they find with a reward higher than the initial action value and then perform that action on all episodes. As a results many of the agents are not able to find the optimal policy.

3.7. Dynamic Programming

Dynamic Programming (DP) refers to a collection of methods used to solve MDP problems. The main idea of DP and RL is the use of value functions to organize and structure the search for good policies. The optimal policies can easily be found once the optimal value functions v_* or q_* which satisfy the Bellman optimality equations shown below are known.

$$\begin{aligned} v_*(s) &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a)\left[r + \gamma v_*(s')\right] \end{aligned} \tag{3.9}$$

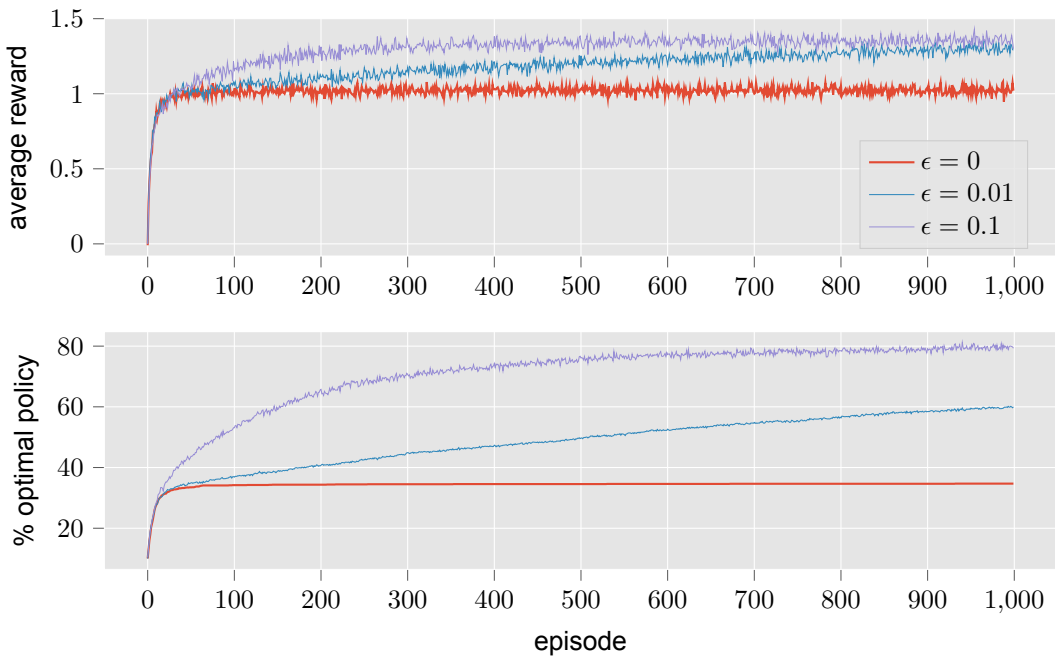


Figure 3.2: Average reward of 2000 10-armed bandit problems for 1000 episodes when following the greedy and two ϵ-greedy policies.

$$
\begin{aligned}
q_*(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a\right] \\
&= \sum_{s',r} p(s', r|s, a)\left[r + \gamma \max_{a'} q_*(S_{t+1}, a')\right]
\end{aligned} \tag{3.10}
$$

DP algorithms are found by turning the Bellman equations into assignment and, or update rules for improving approximations of the value functions.

3.7.1. Policy evaluation

The first step in DP is to compute the *state-value* function v_π for an arbitrary policy π. This is called *policy evaluation* or *prediction problem*.

This can be done by writing the state-value function v_π from eq. 3.6 in terms of the policy probability function $\pi(a|s)$ and the transition probability function $p(s', t|s, a)$. The policy probability function is the probability of action a being selected by the policy while being in state s. And the transition probability function gives the probability of the environment transitioning to state s' and giving reward r if action a is taken while in state s.

$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi\left[G_t | S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1} | S_t = s\right] \\
&= \mathbb{E}_\pi\left[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s\right] \\
&= \sum_a \pi(a|s) \sum_{s',r} p(s', t|s, a)\left[r + \gamma v_\pi(s')\right]
\end{aligned} \tag{3.11}
$$

If the environment's dynamics are known, then Eq. 3.11 is a system of $|\mathcal{S}|$ equations with $|\mathcal{S}|$ unknowns. Solving it would be tedious, but trivial. However, this is almost never the case, thus iterative solution methods provide a better approach. Consider a sequence of value functions $v_1, v_2, v_3, \ldots$, each mapping $\mathcal{S}^+$ to $\mathbb{R}$. The initial value function approximation may be chosen arbitrarily and new approximations are calculated using the Bellman equation for v_π as an update rule.

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}\left[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_k(s')\right] \end{aligned} \tag{3.12}$$

The state-value function v_π is a fixed point for this update rule because the Bellman equation for v_π assures us of equality in this case. It is also possible to show that the sequence v_k approaches v_π as $k \to \infty$. This algorithm is called the *iterative policy evaluation.*

To produce each approximation the iterative policy evaluation applies the operation to each state s and replacing the old value of s with a new one based on the immediate received reward and the old values of the successor states of state s. This operation is called an *expected update*.

One way to implement this algorithm is to keep two arrays, one to hold the old values and another for the new ones. This way the new values can be computed from the old ones, without changing the old ones. It is also possible to implement it with one array and update the values "in place", by immediately replacing the old value by the one. The in-place algorithm not only consumes less memory, but also usually converges faster than the two-array version. This is because the new values are used as soon as they are available.

3.7.2. Policy improvement

With the value function $v_\pi(s)$ for a deterministic policy π known, it is possible to check for each state, whether there is an action a which yields a larger value than the action taken by the current policy π. One way to do this, is for each state s, select an action a different from the action taken by the current policy ($a \neq \pi(s)$) and calculate the value if that action is taken first, followed by the current policy after that. The value for this behavior is:

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_\pi\left(s'\right)\right] \end{aligned} \tag{3.13}$$

If this value is greater than that of the current policy, then a better action has been found. It can therefore be expected that a policy where the new action a is taken whenever state s is encountered is a better policy than the current policy. Thus updating the policy π with the new action will result in a new policy that is as good as or better than the current policy.

Repeating this process for all state-action combinations, it is possible to find a new greedy policy π', given below. Where $\arg\max_a$ denotes the value of a which maximizes the expression that follows.

$$\begin{aligned} q_\pi(s, a) &\doteq \arg\max_a \; q_\pi(s, a) \\ &= \arg\max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_\pi\left(S_{t+1}\right) | S_t = s, A_t = a\right] \\ &= \arg\max_a \; \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_\pi\left(s'\right)\right] \end{aligned} \tag{3.14}$$

The process of making a new policy that improves over the original one using the values of the original policy is called *policy improvement*. In the case where the new policy π' is as good than the old policy π, Then $v_{\pi'} = v_\pi$ and from there is follows that for all $s \in \mathcal{S}$:

$$\begin{aligned} v_{\pi'}(s) &= \max_a \; \mathbb{E}\left[R_{t+1} + \gamma v_{\pi'}(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r|s, a) \left[r + \gamma v_{\pi'}(s')\right] \end{aligned} \tag{3.15}$$

This is the same as the Bellman optimality equation (3.9) and thus $v_{\pi'}(s)$ is the optimal value function $v_*(s)$. So policy improvement will always give a better policy, except when the policy is already optimal.

Policy improvement can also be applied to stochastic policies. These are policies where each action is given a probability $\pi(a|s)$ to occur. If there are ties between actions, that is, multiple actions that result in the same maximum value, then each of these action can be given a part of the probability to occur, as long as any non-optimal action is given zero probability.

3.7.3. Policy iteration

Using policy improvement a policy, π, can be improved using v_π to yield a better policy, π'. With this new policy a new *state-value* function v'_π can be evaluated from π', which results a new improved policy π''. This process can be repeated indefinitely.

Each new policy in guaranteed to be in improvement over the previous one unless it's already the optimal policy. Since a finite MDP only had a finite number of policies, this process must converge to the optimal policy and the optimal value function in a finite number of iterations.

This way of finding the optimal policy it called policy iteration.

3.7.4. Generalized Policy Iteration

In Policy Iteration the Policy Improvement and Policy Evaluation processes alternate, each completing before the other one. However this is not necessary, as long both processes continue to update all states, the ultimate result is almost always the same, convergence to the optimal value function and optimal policy. The term Generalized Policy Iteration (GPI) is used to refer the general idea of letting the Policy Improvement and Policy Evaluation processes interact independent of the granularity and other details of the other processes.

3.8. Monte Carlo Methods

It is not always possible to have a model fully describing the system for which the optimal policy is being discovered. This is where Monte Carlo methods come into play. Unlike Dynamic Programming, Monte Carlo methods require only *experience*, sample sequences of states, actions and rewards or simulated interaction with the environment. Thus no prior knowledge of the dynamics are needed to determine the optimal policy. These methods allow learning from simulated experience, which would require a model, but this model only needs to generate sample transitions and not the complete probability distribution of all possible transitions.

Just like the k-armed bandit problem demonstrated in Section 3.6, Monte Carlo methods work by averaging the returns for state—action pairs. The main difference is that there are multiple states, each acting like a different, interrelated bandit problems, that is, the return from taking an action in one state depends on the actions taken in future states in the same episode.

Since the agent is learning and changing its policy throughout the process, the action selection is constantly changing and thus the problem is nonstationary from the point of view of the earlier state. To handle the nonstationarity, the idea of Generalized Policy Iteration (GPI) is adapted. In DP the value functions are computed directly from the knowledge of the MDP. With Monte Carlo methods however the value functions are *learned* from sample returns with the MDP. The value functions and their policies still interact in the same way.

As previously mentioned, the value of a state ($v_\pi(s)$) is the expected return, the expected cumulative future discounted reward, starting from that state. The obvious way to estimate it from experience is to average all returns after visiting that state. As more returns are observed, the average will converge to the expected value. This is the underlying idea of all Monte Carlo methods.

If no model is available, then it might be better to estimate *action* values (the values of state—action pairs, $q_\pi(s,a)$). With a model, state values alone are sufficient to determine a policy, since the agent would simply have to look ahead one step and see which action results in the highest value. This is not possible without a model. The values of each action must be estimated, since the agent doesn't know to witch state each action will lead to.

3.9. Temporal Difference

Temporal-Difference (TD) learning is one of the main elements of reinforcement learning. It is a combination of ideas from Monte Carlo methods and DP. Just as with Monte Carlo methods, Temporal-Difference (TD) methods are able to learn from experience without the need of a model of the environment's dynamics, and just as with DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome.

3.9.1. TD Prediction

Both TD and Monte Carlo methods use experience to solve the prediction problem. Given some experience following policy π, both methods update their estimate V of v_π for non-terminal states that occurred during that experience. Monte Carlo methods wait until the return G_t after visiting state S_t is known. A simple every-visit Monte-Carlo method is shown in Equation 3.16.

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t - V(S_t)\right] \tag{3.16}$$

Monte-Carlo methods need to wait until the end of an episode since G_t is only known then. TD methods only need to wait until the next iteration by estimating G_t using the observed reward R_{t+1} and the estimated value $V(S_{t+1})$. Thus just like with DP, TD methods updates are based in part on existing estimates. These types of methods are known as bootstrapping methods.

Equation 3.17 shows the simplest TD method. This method is called TD(0) or one-step TD, because it's a special case of the TD(λ) and n-step TD methods.

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right] \tag{3.17}$$

The quantity within the bracket is sort of an error, it's the difference between the estimated value of S_t and the better estimate $R_{t+1} + \gamma V(S_{t+1})$. This quantity is known as the TD error and can be found in various forms throughout reinforcement learning.

3.9.2. Sarsa: On-policy TD Control

In order to use TD prediction for the control problem the pattern of Generalized Policy Iteration (GPI) can be used, only this time using TD methods for the evaluation or prediction part. Approaches fall under two classes, on-policy and off-policy. This section focuses on an on-policy TD control method.

The first step is to learn an action-value function rather than a state-value function. For an on-policy method the estimated $q_\pi(s, a)$ is made for the behavior of the current policy. This can be done essentially the same as the TD method described above.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right] \tag{3.18}$$

This update is done after every iteration from a non-terminal state S_t. If S_{t+1} is a terminal state, then $Q(S_{t+1}, A_{t+1})$ is defined as zero. This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state-action to another. This quintuple gives rise to the name Sarsa for the algorithm.

3.9.3. Q-learning: Off-policy TD control

Another option is Q-learning, an off-policy TD control method. In this case the learned action-value function, directly approximates q_*, the optimal action-value function.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{3.19}$$

The advantage of this method is that it drastically simplifies the analysis of the algorithm and enables early convergence proofs. The only thing that is required is that all pairs continue to be updated. The

disadvantage is that since the action-value function learned is independent of the policy, the agent doesn't take into account the possibility of it taking a non-optimal action.

The following example is used to demonstrate the difference between on and off policy methods. Figure 3.3 shows a grid world. The agent's task to go from the start (S) block to the goal (G). However if the agent steps on the cliff, the episode will end, it receives a -100 reward and is then send back to the start. A -1 reward is given to agent each step until it has reached the goal.

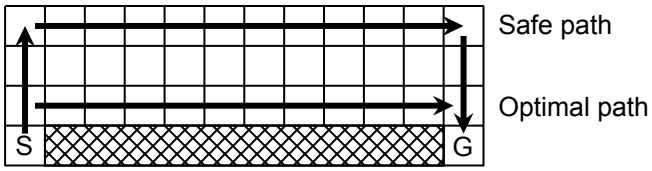


Figure 3.3: Grid world with the safe and optimal paths learned by on-and off-policy methods, where the hashed area is the cliff.

200 agents are simulated for 500 episodes to demonstrate the difference between on and off policy learning methods. They all follow ϵ-greedy policies with $\epsilon = 0.1$, but half use Sarsa and the other half Q-learning. The Q-learning agents learn the optimal policy and thus travel right along the edge of the cliff. However since the agents may at any point take a step into any random direction, it is better to take the safer route further away from the edge. Since the Sarsa agents learn the roundabout policy, this is the path they learn to take. Figure 3.4 show the results of these simulations.

However once the agents have been trained and ϵ is set to zero, the Q-learning agents outperform the Sarsa agents. With these new greedy policies the Q-learning agents will always receive a sum of rewards of -12 while the Sarsa agents receive -16 thanks to the longer path. If ϵ is gradually reduced to zero, then both methods converge to the optimal policy.

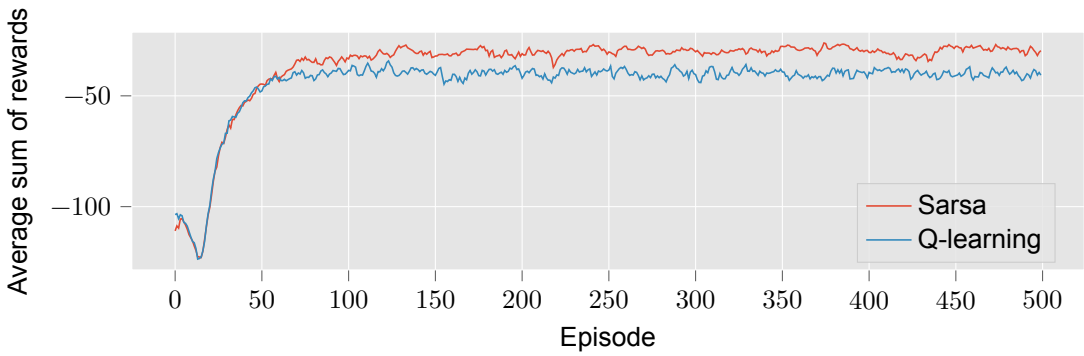


Figure 3.4: Example reward distribution for a 5-armed bandit problem.

3.9.4. n-step Bootstrapping

Consider the expected return used with Monte Carlo methods shown below. It's based on the rewards starting from time t until the end of the episode. Thus while the update does include information throughout a long period of time, it's only possible to update the values after the episode has ended.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + ... + \gamma^{T-t-1} R_T \tag{3.20}$$

In contrast the expected return for one-step TD only uses single reward after one time-step. For many applications it's desirable to update as often as possible, but bootstrapping works best over a period of time after significant and recognizable change has occurred. With one-step TD methods these two lengths of time are the same, so a compromise must be made.

$$G_{t:t+1} \doteq R_{t+1} + \gamma V_t(S_{t+1}) \tag{3.21}$$

While both of these methods are effective in solving MDP problems, the best solutions are usually somewhere in between. n-step Bootstrapping methods provide a spectrum of methods with Monte Carlo in one end and one-step TD on the other by enabling bootstrapping to occur over multiple time steps.

An example family of n-step Bootstrapping methods are n-step TD Prediction methods. For these methods the n*-step return* is as follows,

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + ... + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n}) \tag{3.22}$$

As it is clear from the Equation 3.22, the expected return uses the reward of multiple steps, thus allowing the value function to be updated on each time-step based on information collected over a longer period of time.

3.10. Approximate solutions

Up until now all methods discussed used tables to store the learned value functions and policies. However these methods are not practical for many of the tasks where Reinforcement Learning (RL) is applied due these having large and continuos state spaces.

The problem with large state spaces is not just the memory needed to store the table, but also the time and amount of data needed to fill in these tables accurately. For many of these tasks the states encountered will never have been seen before, thus to make decisions it is necessary to generalize from previous encounters with different states that are similar to the current one. In other words the key issue is that of *generalization*. How can experience with a limited subset of states be generalized to produce a good approximation of a much larger subset?

Fortunately, generalization from examples has been extensively studied, and thus the problem to be solved is combining existing generalization methods with RL methods. The type of generalization methods required are often called *function approximation* because it takes in samples of the target function and creates an approximation of the entire function.

3.10.1. Stochastic Gradient Decent methods

Stochastic Gradient Decent (SGD) methods are a class of function approximation learning methods well suited for the field of Reinforcement Learning (RL). In these methods the weight vector is a real valued column vector with a fixed number of elements, $\mathbf{w} = (w_1, w_2, ..., w_n)^T$, and the approximate function $\hat{v}(s, \mathbf{w})$ is differentiable w.r.t. all weights in $\mathbf{w}$ at all states $s \in \mathcal{S}$.

The goal is to update the weights in $\mathbf{w}$ after each observation such that the approximate function $\hat{v}(s, \mathbf{w})$ better approximates the real function $v_\pi(s)$. This can be done by updating the weights in the direction that would reduce the error the most. A definition of this error is thus needed, one possibility is to use the *Mean Squared Value Error* $\overline{\text{VE}}$ shown below [51].

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w})\right]^2 \tag{3.23}$$

Here is $\mu(s)$ the *state weight distribution*, it is used to indicate which states are more important. This may be necessary since the number of weights in $\mathbf{w}$ can be significantly smaller than the total number of states, thus prioritizing certain states may be needed.

The following update rule for the weights in $\mathbf{w}$ may be defined, using $\overline{\text{VE}}$ and assuming the probability for each state to appear is uniformly distributed.

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t - \frac{1}{2}\alpha \nabla \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w})\right]^2 \tag{3.24}$$

$$= \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w})\right] \nabla \hat{v}(S_t, \mathbf{w}) \tag{3.25}$$

SGD methods are *Gradient Decent* because the updates to the weight vector $\mathbf{w}$ are proportional to the negative gradient of each sample's error. This is the direction where error falls the fastest. Gradient decent methods are called *stochastic* when the updates are done using single samples which may have been selected stochastically. In these cases the updates apply small corrections to the weights and over the course of may iterations it has the effect of reducing the error.

The real value function $v_\pi(s)$ is in practice often not known. Thus the Stochastic Gradient Decent (SGD) may be generalized by substituting it with U_t. This not the true value, $v_\pi(S_t)$, but an approximation of it. It may for example be a noise-corrupted version of $v_\pi(S_t)$, or one of the bootstrapping targets for $\hat{v}$ (eg. G_t, $G_{t:t+n}$, etc).

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\left[U_t - \hat{v}(S_t, \mathbf{w})\right]\nabla\hat{v}(S_t, \mathbf{w}) \tag{3.26}$$

3.10.2. Adam

In SGD the step size parameter, α, has a significant effect on the learning performance. If it's too small, it will learn slowly, if it's too large the target function will not be properly approximated. The step size is also kept constant throughout the learning process and the same value is used to update all parameters in the weight vector, $\mathbf{w}$. This is not ideal, since it may in some cases be beneficial to have separate learning rates for each weight and to update these rates as the agent explores the environment and learns.

Adam is an extension to SGD that uses estimates of the gradient's average 1st moment, better known as the mean, and the gradient's average 2nd moment, better known as the uncentered variance, to adapt the learning rate of each individual parameter in the weight vector.

Algorithm 1: Adam algorithm. Good default values for the hyperparameters are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element wise. With β_1^t and β_2^t being β_1 and β_2 to the power of t[17].

Require: α: Stepsize
Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates
Require: $f(\mathbf{w})$: Stochastic objective function with parameters $\mathbf{w}$
Require: $\mathbf{w}_0$: Initial parameter vector
$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize time step)
while $\mathbf{w}_t$ *not converged* **do**
  $t \leftarrow t + 1$
  $\mathbf{g_t} \leftarrow \nabla_w f_t(\mathbf{w}_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
  $\mathbf{m}_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t$ (Update biased first moment estimate)
  $\mathbf{v}_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_t^2$ (Update biased second moment estimate)
  $\widehat{\mathbf{m}_t} \leftarrow \mathbf{m}_t/(1 - \beta_1^t)$ (Compute biased-corrected first moment estimate)
  $\widehat{\mathbf{v}_t} \leftarrow \mathbf{v}_t/(1 - \beta_2^t)$ (Compute biased-corrected second raw moment estimate)
  $\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \alpha \cdot \widehat{\mathbf{m}_t}/(\sqrt{\widehat{\mathbf{v}_t}} + \epsilon)$ (Update parameters)
end
return $\mathbf{w}_t$ (Resulting parameters)

The algorithm updates exponential moving averages of the gradient, $\mathbf{m}_t$, and the squared gradient, $\mathbf{v}_t$, where the hyperparameters β_1 and β_2 control the exponential decay. These moving averages are themselves estimates of the 1st and 2nd average moments. These vectors are initialized at zero, thus the estimates are biased towards zero, specially during the initial timesteps and when the exponential decays are close to one. This initialization bias is then counteracted yielding the bias-corrected estimates $\widehat{\mathbf{m}_t}$ and $\widehat{\mathbf{v}_t}$.

An important property of the update rule is its careful choice of stepsizes. Assuming $\epsilon = 0$, the effective step taken by each parameter at timestep t is $\Delta_t = \alpha \cdot \widehat{\mathbf{m}_t}/\sqrt{\widehat{\mathbf{v}_t}}$. This effective stepsize has two upper bounds: $|\Delta_t| \leq \alpha \cdot (1 - \beta_1)/\sqrt{1 - \beta_2}$ in case $(1 - \beta_1) > \sqrt{1 - \beta_2}$, and $|\Delta_t| \leq \alpha$ otherwise.

The first case only happens in severe cases of sparsity. When a gradient has been zero for all timesteps except the current one. For less sparse cases, the effective stepsize is smaller. When $(1-\beta_1) = \sqrt{1-\beta_2}$ then $|\widehat{\mathbf{m}_t}/\sqrt{\widehat{\mathbf{v}_t}}| < 1$ therefore $\Delta_t < \alpha$. In most cases $\widehat{\mathbf{m}_t}/\sqrt{\widehat{\mathbf{v}_t}} \approx \pm 1$ since $|\mathbb{E}[\mathbf{g}]/\sqrt{\mathbb{E}[\mathbf{g}^2]}| \leq 1$. Therefore the effective stepsize is bounded by the stepsize hyperparameter, α. This can be understood as establishing a *trust region* around the current weight parameter values, beyond which the current gradient estimate does not provide enough information. This makes it easier to choose a value for α since it only indicates an upper bound for the stepsize.

The $\widehat{\mathbf{m}_t}/\sqrt{\widehat{\mathbf{v}_t}}$ may be seen as some sort of a Signal to Noise Ratio (SNR). With a smaller SNR the effective stepsize will be closer to zero. This property is desirable, since a smaller SNR indicates a greater uncertainty on whether the direction of $\widehat{\mathbf{m}_t}$ corresponds to the direction of the true gradient. For example, the SNR becomes close to 0 towards the optimum, thus reducing the effective stepsize, resulting in a form of automatic step size annealing. The effective stepsize is also invariant of the scale of the gradients. Scaling the gradient $\mathbf{g}$ with a factor c will scale $\widehat{\mathbf{m}_t}$ by c and $\widehat{\mathbf{v}_t}$ by c^2, which cancel out in the SNR.

3.10.3. Artificial Neural Networks

Artificial Neural Networks (ANNs) are widely used for nonlinear function approximation in the field of machine learning and beyond. Deeply-layered ANNs have successfully been used in the Reinforcement Learning to solve complex problems.

An ANN consists of interconnected units which have a similar function to a neuron in a nervous system. Figure 3.5 shows a simple Feed Forward Neural Network (FFNN) with three inputs, three hidden layers and two outputs. The layers are connected to with links, with each link having a real-valued weight associated with it. The links are comparable to synaptic connections in a nervous system and the weight are roughly comparable to the efficacy of the synaptic connections.

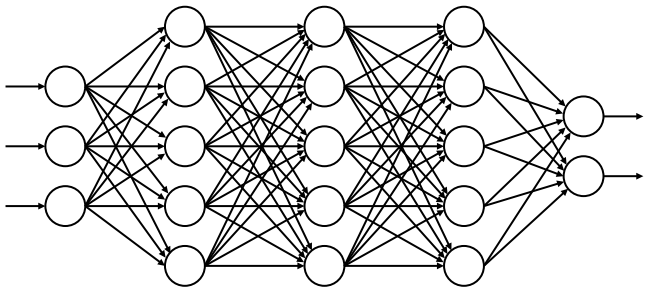


Figure 3.5: Artificial Neural Network[51].

Each one of the units computes its output by first calculating a weighted sum of the inputs and then applies a non-linear function to the result. This non-linear function is called an *activation function*. The activation function is often S-shaped, or sigmoid, for example a hyperbolic tangent, but sometimes a rectifier non-linearity, $f(x) = \max(0, x)$, or step function are also used. It's often useful for different layers to have different types of activation functions.

A FFNN with no hidden layers is only able to approximate linear functions, however if a single hidden layer with a large enough number sigmoid elements is added, then any continuos function may be approximated within a range of input values with any degree of accuracy[6]. This is also true for other non-linear functions that satisfy certain conditions, however non-linearity is essential. Non-linear activation functions are equivalent to a network without hidden layers, since linear functions of linear functions are linear.

Even though a single hidden layer is enough to approximate any function, both practice and theory show that for the complex functions needed in many artificial intelligent tasks it's often easier or may even require abstractions which are hierarchical compositions of many layers of lower-level abstractions, that is, abstractions produced by deep architectures such as ANNs with many hidden layers [1, 51].

3.10.4. LSTM Neural Networks

The network shown in Figure 3.5 is called a Feed Forward Neural Network because it contains no loops, that is, the output of a unit cannot influence the its own input. Networks with loops are called Recurrent Neural Networks (RNNs). An RNN can use the feedback loops to store representations of inputs from recent events. The main advantage of RNNs are the ability to exhibit temporal dynamic behavior, this property allows agents to process sequences of variable length, learn how to behave when there are long intervals of time between events and learn policies for non-Markovian environments.

The Long Short-Term Memory (LSTM) architecture is a type of RNN that is quite popular and has been in use for many years. An LSTM unit consists of a cell, an input gate, an output gate and a forget gate. The cell remembers values for arbitrary time intervals and the three gates regulate the flow of information in and out of the cell[11]. One or more LSTM hidden layers containing an array of these cells may then be used together with feed forward layers to build a deep neural network exhibiting the properties of an RNN.

3.11. Policy Gradient Methods

So far the methods discussed select actions based on estimated (state-)action values, however this is not the only option. Instead of learning the value estimates and deriving a policy out of the resulting value functions, it is also possible to directly learn a *parametrized policy* which can choose actions without having to consult a value function.

Policy gradient methods are a class of learning methods that learn the policy parameters, $\boldsymbol{\theta}$, based on the gradient of some performance measure, $J(\boldsymbol{\theta})$, with respect to the policy parameters. These methods try to maximize the performance, so the updates approximate gradient accent in J.

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} + \alpha \cdot \widehat{\nabla J(\boldsymbol{\theta_t})} \tag{3.27}$$

where α is a positive step-size parameter and $\widehat{\nabla J(\boldsymbol{\theta_t})}$ is a stochastic estimate who's expectation is the gradient of the performance measure with respect to $\boldsymbol{\theta_t}$.

An advantage of using policy gradient over ϵ-greedy action selection is that the approximate policy can approach a deterministic policy if the optimal policy is deterministic, whereas ϵ-greedy action selection will always have an ϵ probability of selecting a random action.

Similarly if the optimal policy is stochastic or in cases where multiple actions have similar values, then policy gradient methods are able to converge towards stochastic policies who's probability distribution approximate the distribution of the optimal policy. In contrast action-value methods are unable to find optimal stochastic policies.

In problems with significant function approximation, the best policy may be stochastic even for deterministic problems. With imperfect information the optimal policy may be to perform two or more different actions with specific probabilities, for example when bluffing in Poker.

This is specially helpful for continuos state and action spaces. Since, as discussed in Section 3.10, for these kind of systems it is usually not practical to store, process and learn policy and value functions based on tables. Instead these functions are usually approximated.

3.11.1. Policy parameterization

In policy gradient methods the policy may be parametrized in any way so long as the policy function, $\pi(a|s,\ \boldsymbol{\theta})$, is differentiable with respect to the policy parameters, $\boldsymbol{\theta}$.

A method of parameterization in case of discreet and small action spaces is to form parametrized numerical preferences $h(s, a, \boldsymbol{\theta}) \in \mathbb{R}$ for each state-action pair. The actions with the highest preference can then be given the highest probability of being selected by, for example, using an exponential softmax distribution,

$$\pi(a|s, \boldsymbol{\theta}) = \frac{\exp(h(s, a, \boldsymbol{\theta}))}{\sum_b \exp(h(s, b, \boldsymbol{\theta}))} \tag{3.28}$$

The denominator is needed such that the probabilities in each state sum to one, thus allowing the preference to be parametrized using any function, whether it is a table, a linear function using feature vectors or a deep neural network.

For problems with large or continuos action spaces it may be better to learn the statistics of the probability distribution instead of the preference of each individual action. For example, the actions may be chosen using a normal (Gaussian) distribution. In this case the mean, μ, and standard deviation, σ, of the distribution may be parametrized using any function approximation method, with the only limitation being that the function for the standard deviation must always be positive.

$$\pi\left(a|s,\boldsymbol{\theta}\right)=\frac{1}{\sigma(s,\boldsymbol{\theta})\sqrt{2\pi}}\exp\left(-\frac{(a-\mu(s,\theta))^2}{2\sigma(s,\theta)^2}\right) \tag{3.29}$$

3.11.2. Policy gradient theorem

The goal with MDP problems is to find a policy which maximizes the sum of rewards the agent receives. The value function approximates an expectation of the sum of rewards, thus the perform measure, $J\left(\boldsymbol{\theta}\right)$, can be assumed as being approximately equal to the true value function under current policy π_θ.

$$J\left(\boldsymbol{\theta}\right)\doteq v_{\pi_\theta}\left(s\right) \tag{3.30}$$

The problem with finding a change in the policy parameters that ensures improvement is that the performance depends on both the action selection and the distribution of states. At a given state, estimating the effect on the actions, and thus the reward, is relatively straightforward. However the effect of the policy on the state distribution depends on effects from the environment which are typically unknown.

Fortunately, the *policy gradient theorem* provides an analytical expression that is proportional to the gradient of the performance with respect to the policy parameters, which doesn't involve the derivative of the state distribution[51].

$$\nabla J\left(\boldsymbol{\theta}\right)\propto\sum_s\mu\left(s\right)\sum_a q_\pi\left(s,a\right)\nabla_\theta\pi\left(a|s,\boldsymbol{\theta}\right) \tag{3.31}$$

3.11.3. REINFORCE

Recall from Equation 3.27 the overall strategy for policy gradient methods requires an estimate of the gradient of the performance measure with respect to the policy parameters. This estimate only needs to be proportional to the real gradient, since any constant of proportionality can be absorbed by the step size α, which is otherwise arbitrary. The policy gradient theorem (Equation 3.31) gives an expression for such an estimate.

The right-hand side of the policy gradient theorem is a sum over the states weighted by how often the states occur under the target policy π. If π is followed, then the states will be encountered in these proportions. Thus,

$$\nabla J\left(\boldsymbol{\theta}\right)\propto\sum_s\mu\left(s\right)\sum_a q_\pi\left(s,a\right)\nabla_\theta\pi\left(a|s,\boldsymbol{\theta}\right) \tag{3.32}$$

$$=\mathbb{E}_\pi\left[\sum_a q_\pi\left(S_t,a\right)\nabla_\theta\pi\left(a|S_t,\boldsymbol{\theta}\right)\right] \tag{3.33}$$

The remaining part of the expectation is a sum over the actions. If each term was weighted by the probability of selecting the actions, that is by, $\pi\left(a|S_t,\theta\right)$, then it is possible to replace a by the sample action A_t. This can be done by multiplying and dividing by this probability.

$$\nabla J(\boldsymbol{\theta}) = \mathbb{E}_{\pi}\left[\sum_{a} \pi(a|S_t,\theta)\, q_{\pi}(S_t,a)\, \frac{\nabla_{\theta}\pi(a|S_t,\boldsymbol{\theta})}{\pi(a|S_t,\theta)}\right] \tag{3.34}$$

$$= \mathbb{E}_{\pi}\left[q_{\pi}(S_t,A_t)\, \frac{\nabla_{\theta}\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\theta)}\right] \quad \text{(replacing } a \text{ by sample } A_t \sim \pi\text{)} \tag{3.35}$$

$$= \mathbb{E}_{\pi}\left[G_t \frac{\nabla_{\theta}\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\theta)}\right] \quad \text{(Because } \mathbb{E}_{\pi}[G_t|S_t,A_t] = q_{\pi}(S_t,A_t)\text{, Equation 3.7)} \tag{3.36}$$

Substituting this expression into Equation 3.27 yields the update rule,

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} + \alpha G_t \frac{\nabla_{\theta}\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\theta)} \tag{3.37}$$

This is the REINFORCE algorithm. Note that REINFORCE uses the complete return from t until the end of the episode. In this sense REINFORCE is a Monte Carlo algorithm and is thus only well defined for episodic problems.

Each update is proportional to the product of the return, G_t, and a vector in the direction that most increases the probability of taking A_t on future visits to S_t. This vector is the *eligibility vector*. The eligibility vector is usually written in the more compact form $\nabla_{\theta} \ln \pi(A_t|S_t,\boldsymbol{\theta})$, using $\nabla \ln x = \frac{\nabla x}{x}$. Thus the REINFORCE algorithm may also be written as,

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} + \alpha G_t \nabla_{\theta} \ln \pi(A_t|S_t,\boldsymbol{\theta}) \tag{3.38}$$

3.11.4. REINFORCE with baseline

The policy gradient theorem can be generalized by including a comparison of the action value to an arbitrary baseline $b(s)$.

$$\nabla J(\boldsymbol{\theta}) \propto \sum_{s} \mu(s) \sum_{a} (q_{\pi}(s,a) - b(s))\, \nabla_{\theta}\pi(a|s,\boldsymbol{\theta}) \tag{3.39}$$

This baseline can be any function, even a random variable, as long as it does not vary with the action[51]. Following a similar procedure as in the previous section yields the following update rule for REINFORCE with baseline,

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} + \alpha (G_t - b(S_t)) \frac{\nabla_{\theta}\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\theta)} \tag{3.40}$$

The baseline leaves the expected value of the update unchanged, however it can have a large effect on its variance and thus the learning rate. An appropriate choice for the baseline is an estimate of the state-value function $\hat{v}(S_t)$[51]. In this case, the term with the expected return and baseline value function is known as the expected advantage function.

$$A_t = G_t - \hat{v}(S_t) \tag{3.41}$$

Since REINFORCE is a Monte Carlo method for learning the policy parameters θ, it seems appropriate to use the similar Stochastic Gradient Decent (SGD) function approximation learning method shown in Section 3.10.1 to learn the value function with weights $\mathbf{w}$.

3.11.5. Actor-Critic

Methods that learn both the policy and value functions are called *actor-critic* methods, where the actor refers to the learned policy, and the critic to the learned value function. However REINFORCE with baseline is not considered an actor-critic method because the state-value function is only used as a baseline and not a critic. That is, it is not used for *bootstrapping*, the process of updating the value estimate of a state using the estimated values of future states, such as with the Temporal-Difference (TD) methods found in Section 3.9.

Consider the one-step actor-critic methods. These methods replace the complete return used in REINFORCE with baseline with the one-step return. Expanding these methods to include n-step bootstrapping is then straightforward.

$$\boldsymbol{\theta}_{\mathbf{t+1}} = \boldsymbol{\theta}_{\mathbf{t}} + \alpha \left(G_{t:t+1} - \hat{v}\left(S_t, \mathbf{w}\right)\right) \frac{\nabla_\theta \pi\left(A_t | S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t | S_t, \theta\right)} \tag{3.42}$$

$$= \boldsymbol{\theta}_{\mathbf{t}} + \alpha \left(R_{t+1} + \gamma V_t(S_{t+1}) - \hat{v}\left(S_t, \mathbf{w}\right)\right) \frac{\nabla_\theta \pi\left(A_t | S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t | S_t, \theta\right)} \tag{3.43}$$

$$= \boldsymbol{\theta}_{\mathbf{t}} + \alpha \delta_t \frac{\nabla_\theta \pi\left(A_t | S_t, \boldsymbol{\theta}\right)}{\pi\left(A_t | S_t, \theta\right)} \tag{3.44}$$

3.11.6. PPO

A limitation of standard policy gradient methods is that it is hard to choose step sizes when updating the policy. This is in part because a bad update step is quite damaging since it may reduce the visitation distribution around the state. Stepping to far results in a bad policy, which leads to new data being collected under the bad policy, which can cause performance collapse. Another issue with standard policy gradient methods is that these methods have low sample efficiency, only one gradient step is performed per environment sample[47, 48].

So, how can the biggest improvement step on the policy be taken without over stepping and causing performance collapse? This is the question that methods such as Trust Region Policy Optimization (TRPO), and Proximal Policy Optimization (PPO) seek to answer.

In TRPO a surrogate objective function is maximized. This function is an approximation of the expected advantage function of the current or new policy. It is the expected advantage from the old policy being used when the sample was taken and then calibrated using the probability ratio between the new and old policies. This approximation becomes less accurate, as the two policies deviate from each other.

This objective function is then constrained by the KL-divergence of the new and old policies. The KL-divergence function measures the difference between two probability distributions. This problem is then solved using the conjugate gradient algorithm after making a linear approximation of the objective function and a quadratic approximation of the constraint [47].

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \mathbb{E}_{\mathbb{t}}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t\right] \\ \text{subject to} \quad & \mathbb{E}_{\mathbb{t}}\left[\overline{KL}\left[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)\right]\right] \leq \delta \end{aligned} \tag{3.45}$$

x sc The theory justifying TRPO actually suggests using a penalty instead of a constraint as shown in Equation 3.46. In this case the objective function computes the maximum KL-divergence over the states and subtracts it from the expected advantage estimate,forming a lower or pessimistic bound on the performance of the new policy, π. This lower bound function can then be used to optimize the policy for maximum rewards using the Minorize-Maximization (MM) algorithm shown in Figure 3.6. MM works by iteratively maximizing a lower bound function which locally approximates the target function.

$$\underset{\theta}{\text{maximize}} \quad \mathbb{E}_{\mathbb{t}}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t - \beta\, \overline{KL}\left[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)\right]\right] \tag{3.46}$$

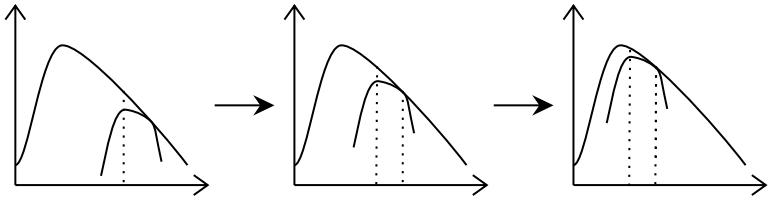


Figure 3.6: Minorize-Maximization algorithm

The reason TRPO uses a hard constraint instead of the penalty is because it is difficult to choose a value for β that works well between problems or even within the same problem. However PPO provides new alternate surrogate objective functions that work around this issue. Despite PPO not being as good as TRPO in determining an appropriate size for the trust region, it is much simpler to implement and less computationally intensive. Benchmarks show that PPO is able to perform as good as or even better than other state-of-the-art methods[48].

There are two variants of PPO. PPO with adaptive KL penalty and PPO with clipped objective. This thesis only uses PPO with clipped objective, thus the first variant is left out.

Let $r_t(\boldsymbol{\theta})$ be the probability ratio $r_t(\boldsymbol{\theta}) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. Then the policy surrogate objective function proposed by PPO is as follows.

$$L_t^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbb{t}}\left[\min(r_t(\boldsymbol{\theta})A_t, \text{clip}(r_t(\boldsymbol{\theta}), 1-\epsilon, 1+\epsilon)A_t)\right] \tag{3.47}$$

Where ϵ is a hyperparameter, in the original paper the value of $\epsilon = 0.2$ was chosen based on benchmarks[48]. The first term inside the $\min(...)$ is the same as the surrogate objective function used in TRPO (Equation 3.45). The second term clip$(r_t(\boldsymbol{\theta}), 1-\epsilon, 1+\epsilon)A_t$ modifies the surrogate objective by clipping the probability ratio, thus removing the incentive for moving r_t outside of the interval $[1-\epsilon, 1+\epsilon]$

If a neural network architecture that shares parameters between the policy and value functions is being used, then a loss function that combines the policy surrogate objective function and a value function error term must be used. An entropy bonus may also be added to encourage exploration. Combining these terms results in the following objective function[48].

$$L_t^{CLIP+VF+S}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbb{t}}\left[L_t^{CLIP}(\boldsymbol{\theta}) + c_1 L_t^{VF}(\boldsymbol{\theta}) + c_2 S[\pi_\theta](s_t)\right] \tag{3.48}$$

Here are c_1 and c_2 coefficients, S the entropy bonus and L_t^{VF} is the squared-error loss of the value function estimate. This objective function can then be optimized on each iteration using SGD or Adam. This allows each environment sample to be used for multiple policy updates, thus resulting in a better sample efficiency than standard gradient decent methods.

3.12. Example: Quadrotor guidence law

The purpose of this example is to gain a better understanding of the basic workings of Reinforcement Learning and how the learning process is affected by the hyperparameters α and γ. The problem, based on Junell et al. [15], consists of a quadrotor agent within a grid world containing three collapsed buildings and an upload station. The goal of the agent is to visit the disaster sites to collect pictures and every once in a while upload its memory contents at the upload station.

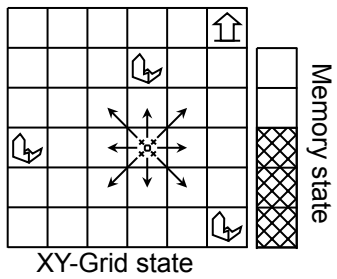


Figure 3.7: Example state and possible actions.

Each time the quadrotor visits a disaster site its memory state is increased by one and the agent is given a reward of 8. In total the agent has the memory capacity of 5. Whenever the agent visits the upload station it is given a reward equal to the square of the memory state and the memory state is reset back to zero. After each visit to either a disaster site or the upload station, the agent is moved to a random

location in the grid world. If the agent leaves the grid world it's given a negative reward and is moved back to its last position on the grid. As possible actions the agent can choose to move to any adjacent grid location including diagonal movements.

The agents use a simple ϵ-greedy on-policy learning method using the update rule shown in Equation 3.49. Figure 3.8 shows the mean rewards of 1000 agents with varying hyperparameters α and γ. As it can be seen decreasing the learning rate α reduces the convergence rate, but results in better learned policies in the long run.

Having $\gamma = 0$ results in poor learning performance, this is because each update is only dependent only on the reward the agent received after performing the action. It doesn't take future action-states into account. In contrast when setting γ to a non-zero value, the update is dependent on the value of the following action-state. Thus giving the agent an indication of the value of future events.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right] \tag{3.49}$$

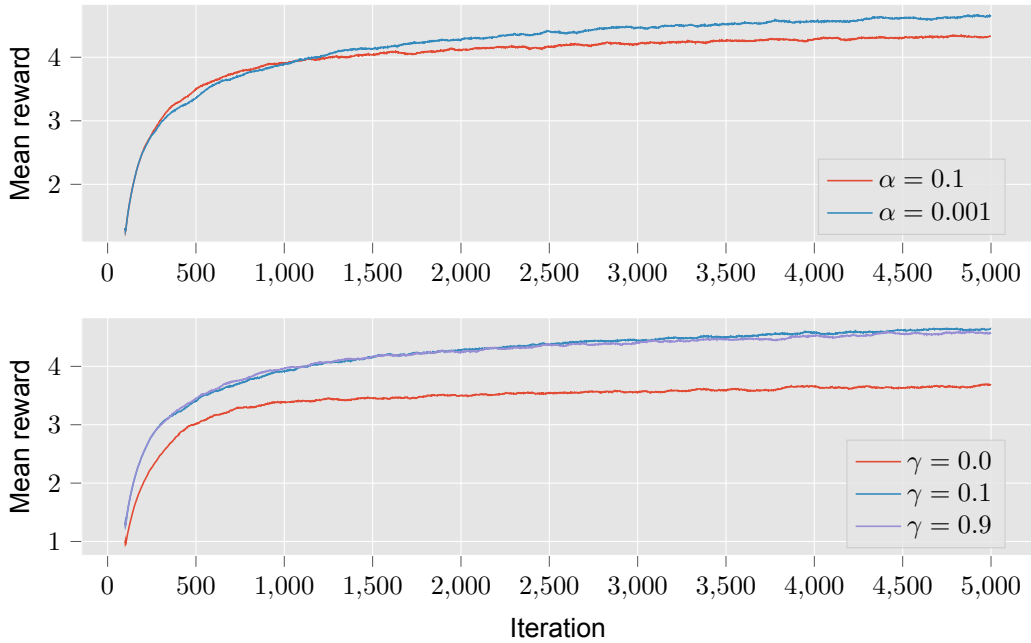


Figure 3.8: Learned policy at each memory state.

Figure 3.9 shows the greedy policy of one of the agents at each memory state after the learning process. Being an ϵ-greedy on-policy learning method it can be seen that the agents avoid traveling along the edges of the grid world, so as to avoid stepping outside during a random action.

It can also been seen that at low memory the agents prefer to travel to the disaster sites and as the memory fills up it becomes more valuable to the agents to travel to the upload station. This is thanks to the increase in rewards the agents are able to get from the upload station as the memory fills up. Once full, the only place where the agents are able to get a reward is from the upload station, so in this case the policy always guides the agent towards the upload station.

3.13. Example: Suborbital flight

The goals of this example are to develop the simulation and to experiment with the learning methods and libraries that will be used throughout the main thesis project.

3.13.1. Simulator

The simulator is implemented in Python using Numba to optimize computation time. It takes place in a 2D world where the vehicle has 3 degrees of freedom, the x and y position and a pitch angle θ. A

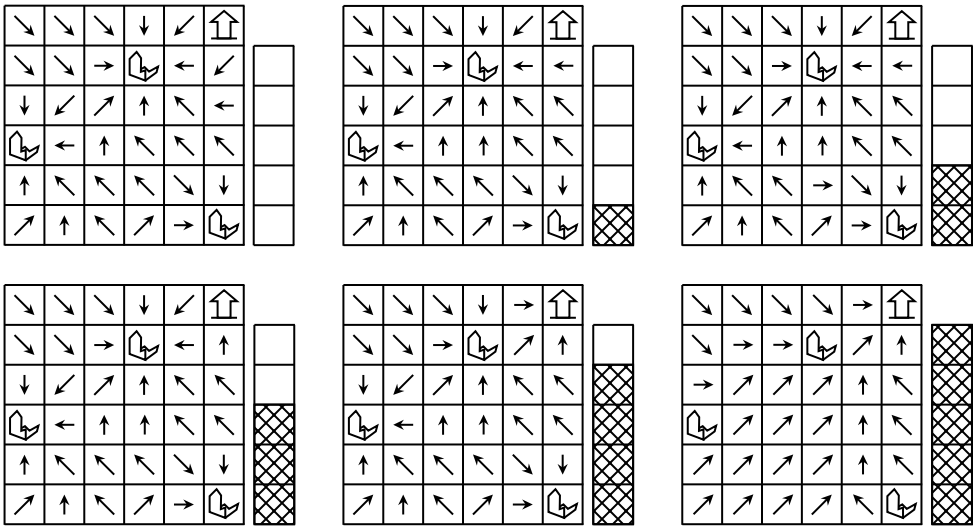


Figure 3.9: Mean rewards with varying learning rate and discount factor.

non-rotating analog of the moon is chosen as the celestial body at which to simulate. The main reasons for this choice are:

1. A smaller celestial body means shorter distances and thus shorter simulation times are needed to ascent to orbit. Thus the learning process should go faster.
2. There already are real world designs of single stage to lunar orbit spacecraft that can be used as examples during early learning runs.
3. No atmosphere.

The reference frames used in the simulation are shown in Figure 3.10. The inertial frame, I, is located at the center of the moon. The vehicle-carried normal lunar reference frame, E, has it's origin at the surface of the moon right under the location of the vehicle, with the y-axis being normal to the surface. The body frame, B, has its origin at the vehicle center of mass and is oriented with the vehicle structure. Furthermore it's assumed to be in vacuum, thus no aerodynamic forces nor moments are included. Later simulations may include an atmosphere.

The set of differential equations are shown in Equation 3.50. These are solved using a third order *Adams–Bashforth* (AB3) integrator. This is an explicit, multi-step integration method that can approximate a third order solution using the results from previous integration steps. Since the previous steps need to be known, a forth order *Runge–Kutta* integrator is used for the first four iterations. The main advantage of using an Adams–Bashforth integrator is that the derivatives only need to be evaluated once per integration step while still yielding higher order results, thus improving computational efficiency while minimizing integration errors.

$$
\begin{aligned}
\dot{x_x^I} &= v_x^I & \dot{v_x^I} &= a_x^I = \frac{F_{T_x}^I}{m} \\
\dot{x_y^I} &= v_y^I & \dot{v_y^I} &= a_y^I = \frac{F_{T_y}^I}{m} \\
\dot{m} &= \frac{|F_T|}{gI_{sp}} & \dot{\theta^I} &\quad \text{(input)}
\end{aligned}
\tag{3.50}
$$

The simulation supports multi-stage launch vehicles. By default the main control input is the pitch angle rate, $\dot{\theta^I}$, but the simulation includes configurable PID controllers for controlling the pitch and flight path angles.

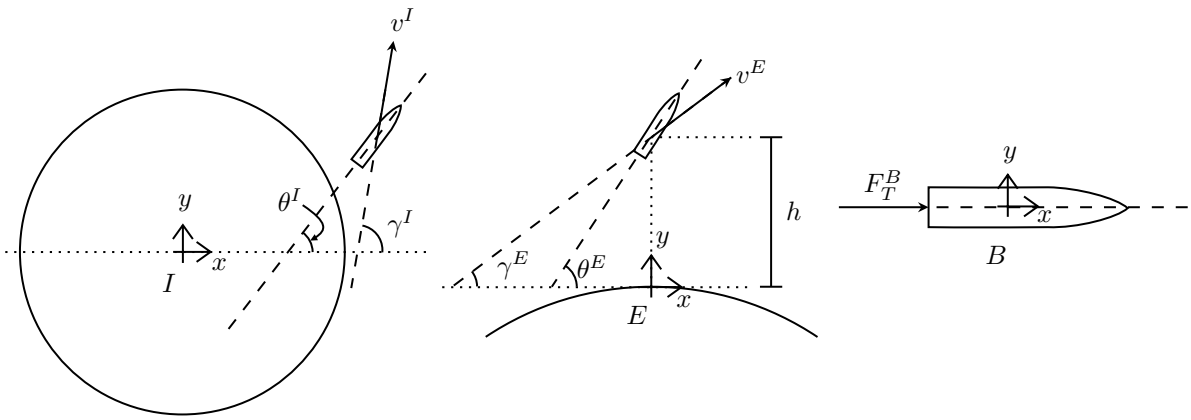


Figure 3.10: Simulation reference frames and states.

3.13.2. The environment

The vehicle used during this example is a small single stage vehicle incapable of reaching orbit. Instead the goal given to the agent is to maximize the apocynthion during a suborbital flight. The vehicle properties are shown in Table 3.1.

m_{dry}	1 kg
m_{prop}	0.2 kg
I_{sp}	80 s
F_T	6.8 N

Table 3.1: Suborbital launch vehicle properties.

In this environment the simulation is configured to end if the vehicle contacts the lunar surface or at engine burnout. The reward is only given at the terminal state and is defined by the following equation.

$$R_{t_{end}} = \left[\frac{h + \frac{v_y^{E2}}{2g}}{4600} \right]^{21} \tag{3.51}$$

The numerator of the outer division is the vehicle altitude plus the altitude it would be able to ascent with the remaining vertical kinetic energy. This gives an estimate of the apocynthion the vehicle can reach with its current state. This estimate is then divided by the maximum apocynthion the vehicle can potentially reach in ideal conditions to get a normalized reward. The reward is then raised to the power of 21 in order to lower the rewards given at low apocynthions, thus encouraging the agent to manoeuvre to higher apocynthions.

The only action available to the agent is a continues pitch rate command between -1 and 1 rad s^{-}1. The observation is a tuple containing $(\gamma^E, \theta^E, \frac{h}{1200})$. The altitude is normalized by dividing by 1200 rad s^{-}1.

The reason why the reward and altitude are normalized is because machine learning libraries tend be optimized or their default hyperparameters are chosen with the assumption that the inputs and outputs are normalized.

Furthermore the vehicle's initial position is always on the north pole and the initial velocity is always zero. The initial pitch angle, θ^E is randomly selected between $[15\,\mathrm{deg}, 165\,\mathrm{deg})$.

3.13.3. The agent

The agent is implemented using RLlib (Liang et al. [19]), a Reinforcement Learning Python library designed to take advantage of parallel computing. It provides a set of primitive abstractions around the different RL components, thus allowing the use of different machine learning frameworks, learning algorithms, environment, etc. RLlib also contains implementations for common RL algorithms which will be used as a basis during this thesis.

An actor-critic agent is used in this example. The model consists of a Feed Forward Neural Network (FFNN) with two hidden layers, each containing 32 neurons. All layers are shared between the policy and value functions and finally the agent is trained using a PPO trainer following the methods described in Section 3.11.6.

As already mentioned RLlib allows the use of parallel computing to accelerate the training process. For the PPO trainer the setup consists of a central learner process and a set a worker processes that run episodes using the latest model while the central learner is updating the model. After the central learner is done updating the models, the workers are updated with the new model and then the simulation results are send to the central learner to repeat the learning process.

This allows the agent to learn from a varied set of episodes during each model update, thus increasing the number of visited states and actions, and prevent one "bad" episode to significantly influence the update.

3.13.4. PID controller

To have something to compare to a controller was developed consisting of a PID outer loop controlling the flight path angle, γ^E, and sending a reference signal to a PID inner loop controller controlling the pitch angle, θ^E, by sending pitch rate commands to the environment. To maximize the apocynthion the controller is given a constant flight path angle command of $\gamma^E = 90°$.

The training processes lasted for 3 hours, 50 minutes and 31 seconds, and ended when the model started outputting NaN values.

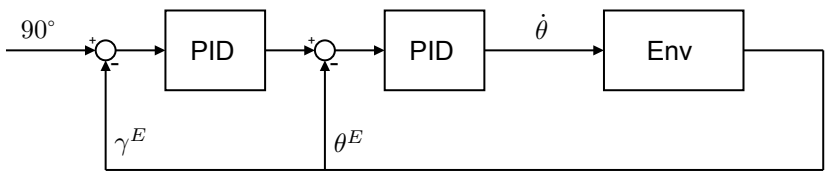


Figure 3.11: PID controller.

3.13.5. Results

In total the agent was trained for 2252 batches, each with on average 12.1 episodes. Figure 3.12 shows the average reward in each batch and the rewards achieved by agents following the greedy policy after each batch at different initial pitch angles θ^E. The figure also shows the rewards achieved by the PID controller.

Figure 3.13 shows the KL-divergence, policy loss and value function loss during each update. As explained in Section 3.11.6 the KL-divergence is a measure of difference between two probability distributions, in this case the policies before and after training. The policy and value function losses are the terms corresponding to the policy and value function errors in the objective function being optimized. Together these three values give an indication for when the agent is learning and updating its model.

From the results it is clear that most of the learning happens within the first 500 batches, after which it remains mostly stagnant. The agent is able to reach and occasionally exceed the reward achieved by the PID controller.

Just after 900 batches, the greedy agent reward dips down and the value function losses increase. This

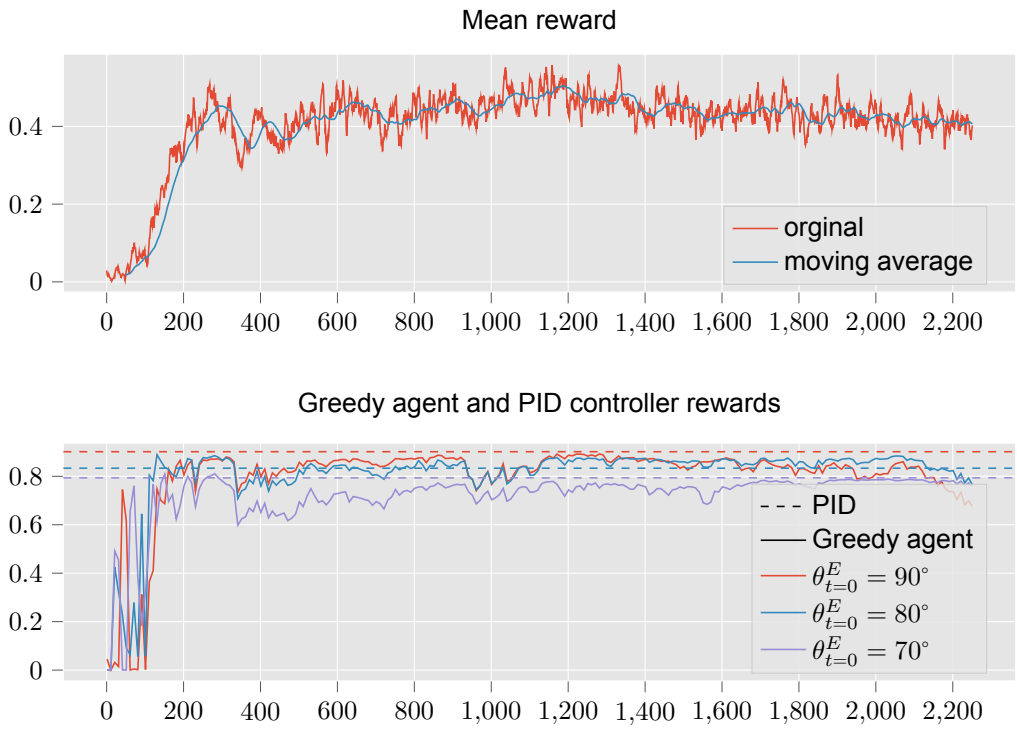


Figure 3.12: Reward history during training.

may be a sign of overfitting, which would also explain why the model returned NaN values on it's last episode.

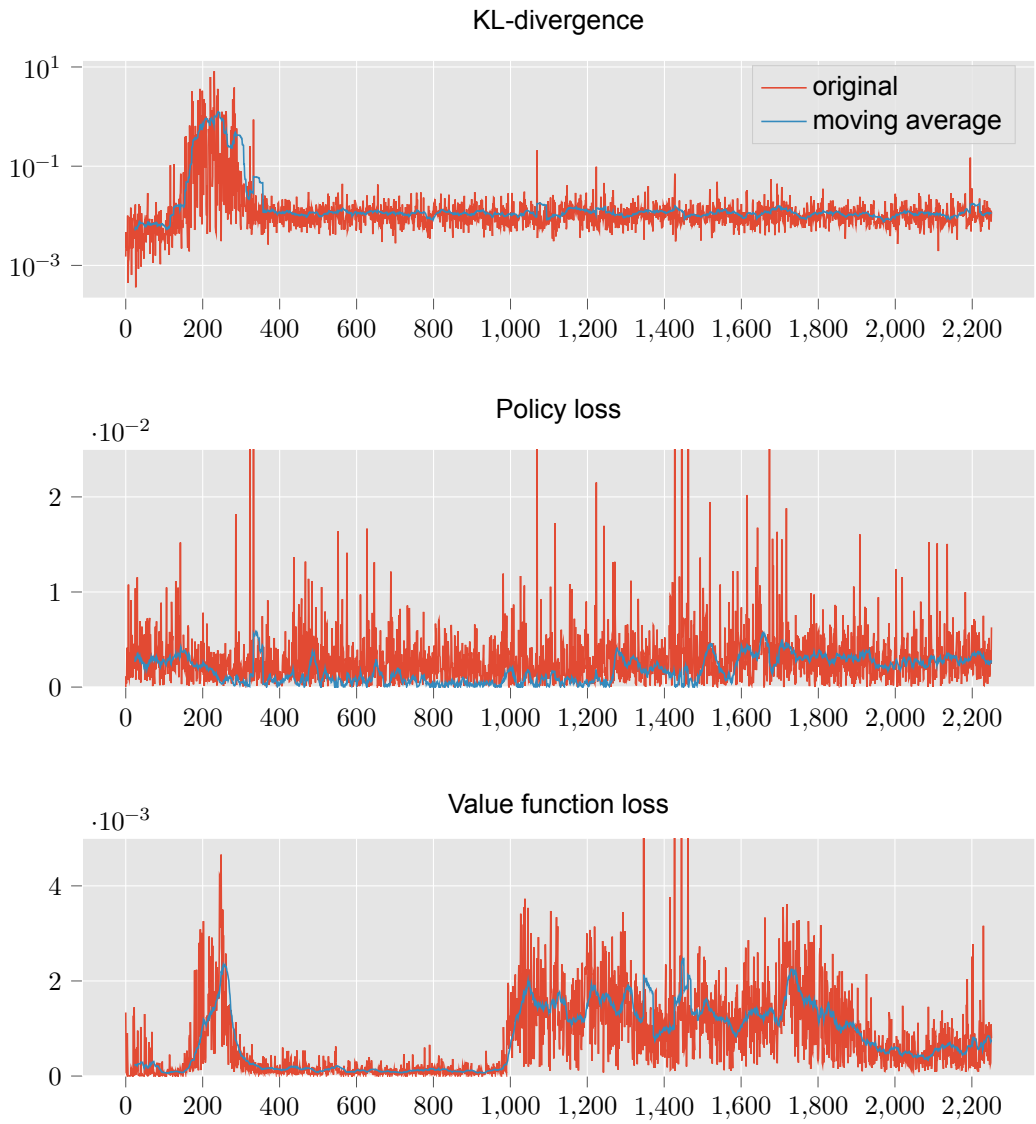


Figure 3.13: KL-divergence, policy and value function losses during training.

4

State of the art

Research into the use of RL in spacecraft guidance is relatively young. The main characteristics of RL that are of special interest in space guidance is the ability to generalize and autonomously adapt to unknown situations that are difficult to forsee or too cumbersome to take into account. This is specially useful for deep space missions due to the unavoidable communication lag which make it difficult for remote human operators to make adjustments in fast changing situations.

4.1. Asteroid hovering

As an example the research by Gaudet and Furfaro [8] explores the use of RL as a control method for hovering in the non-uniform, rotating gravitational field commonly associated with asteroids. Current methods rely on prior knowledge which can be costly to acquire. The goal was to develop and train an RL which can accurately hover in unknown environments.

A 6-DOF simulation was used, with the asteroids assumed to be rotating tri-axial ellipsoids with varying semi-axes lengths and uniform density. A ANN was used to serve as an estimate of the policy function, with the weights being optimized using a gradient-less stochastic search algorithm.

The ability to generalize resulted in robust agents that could accurately hover in environments that where unknown during the policy optimization process. Figure 4.1 show a histogram of the maximum steady state error over 1000 simulations in randomly generated environments.

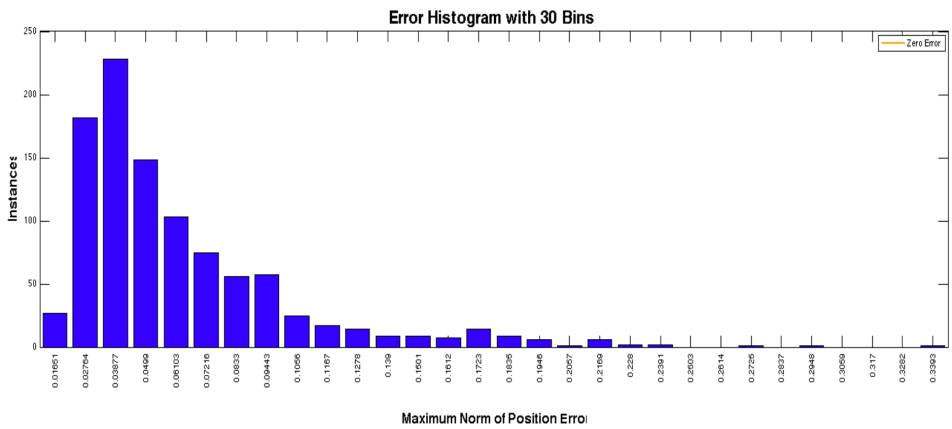


Figure 4.1: Histogram of the maximum norm of the position error over 1000 simulations in meters.[8]

To further test the agent the worst result was simulated to a total simulation time of 10 hours. The maximum deviation from the target position remained unchanged during this longer simulation.

Gaudet and Furfaro also implemented a linear proportional and derivative (PD) controller with adaptive

gains produces by a trained RL agent. The maximum steady state error increased from 0.345 m to 1.819 m and had the tendency to overshoot more often the pure RL solution.

For comparison a Linear Quadratic Regulator (LQR) controller was implemented. The steady errors where similar to that of the RL controller, but the transient response exhibited more oscillations. This is likely due tot he fact that the information about the environments dynamics was contained in the system matrix used to design the LQR.

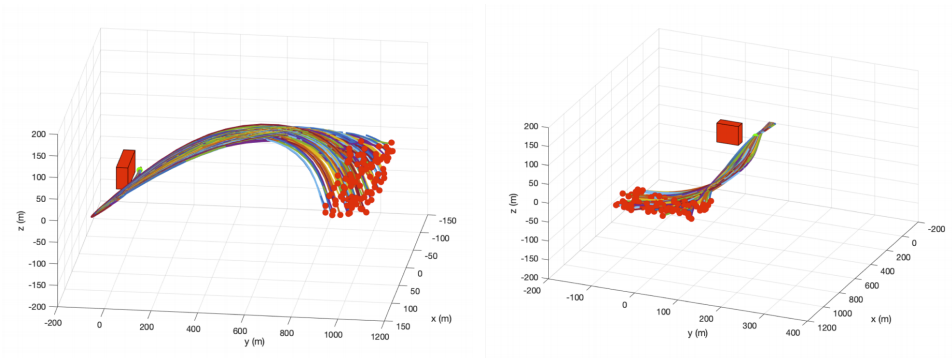
Finally A simulation batch was run using a polyhedron shape of the asteroid Itokawa while varying the initial distance from the target position. The RL controller performed better than with the randomly generated asteroids by showing a maximum steady state error of only 0.0310 m.

Further development by Willis et al. [56] managed to improve the accuracy by an order of magnitude by changing the optimization method to Participle Swarm Optimization (PSO). A more general gravitation model was also used allowing for a wider range of asteroids to be simulated.

4.2. Proximity operation guidance

More recently Hovell and Ulrich [12, 13] and Broida and Linares [3] have been developing RL based control system for proximity operation guidance, such as during autonomous rendezvous and docking operations. These could be applied in situations such in-orbit servicing, assembly, and debris capture, where the chaser spacecraft may be required to manoeuvre in the proximity of a potentially uncooperative target object while avoiding nearby obstacles in cluttered environments.

Broida and Linares simulated relative orbits using Clohessy–Wiltshire (CW) dynamics to train the agent using PPO in two canonical satellite docking scenarios, the R-bar and V-bar approaches. The environment also included a keep-out zone to protect against collisions, and during final testing stochastic disturbances where added to demonstrate the robustness of the trained policy. Figure 4.2 shows the keep out zone and a set trajectories taken by the fully trained agent.



(a) 100 V-bar Trajectories (b) 100 R-bar Trajectories

Figure 4.2: Plotted here are 100 fully trained trajectories for both the V-bar and R-bar approaches [3].

During testing the agent managed to close the distance with a final state error in the order of a few centimeters. The agent also learned to avoid the keep-out zone and successfully avoided it all simulated test cases. Broida and Linares conclude that the results are promising, and that further research could continue towards more complex problems.

Hovell and Ulrich [12] simulate on a planar environment. The agent is tasked to first navigate the chaser spacecraft to a safe hold point, before approaching the target for docking. Figure 4.3 shows a diagram with an example initial position and the target hold and docking points.

D4PG, an actor-critic RL learning method, was used to train the agent to dock with both stationary and rotating targets. In both cases the trained agent managed to consistently complete its tasks accurately during final testing.

To validate the results a practical experiment was also performed. In the experiment the environment was modelled by two air-bearing spacecraft platforms operating on a granite surface to reduce the

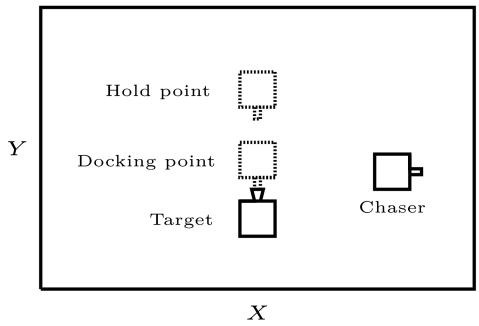


Figure 4.3: Sample spacecraft pose tracking and docking task. [12].

friction to negligible levels and the spacecraft where actuated by small cold gas thrusters. The results of the experiment matched the simulated results, thus demonstrating the proposed technique on real world system.

During further research, Hovell and Ulrich [13] repeated the same simulated and practical experiments but included a stationary obstacle the agent would have to avoid. In all cases the agent was able to successfully rendezvous with the target spacecraft without colliding with the obstacle.

4.3. Precision planetary powered landing

Gaudet et al. [9] propose the use of a PPO RL agent for planetary powered decent and landing. Future interplanetary space missions will require spacecraft to target specific locations on the surface. The aim of this research is to develop a landing system that can land within a few meters from its target. The agent was trained on a 6-DOF Mars landing simulation, and results demonstrated that the goal to achieve pin-point accuracy was reached. Figure 4.4 shows the touchdown statistics for 10000 episodes with the trained agent.

	Mean	Std. Dev.	Min	Max
Downrange Position (m)	0.4	0.7	-2.9	4.5
Crossrange Position (m)	-0.1	0.7	-5.2	2.5
Downrange Velocity (m/s)	0.06	0.03	-0.04	0.14
Crossrange Velocity (m/s)	-0.01	0.04	-0.20	0.14
Elevation Velocity (m/s)	-0.93	0.08	-1.32	-0.49
Pitch (rad)	-0.016	0.010	-0.060	0.028
Roll (rad)	-0.003	0.011	-0.043	0.050
Rot. Velocity - Roll (rad/s)	-0.000	0.021	-0.105	0.084
Rot. Velocity - Pitch (rad/s)	-0.005	0.013	-0.083	0.051
Rot. Velocity - Yaw (rad/s)	0.000	0.000	0.000	0.000
Glideslope	22.02	17.77	7.50	851.54
Fuel Consumed - 6-DOF RL Policy (kg)	291	15	257	352
Fuel Consumed - 3-DOF RL Policy (kg)	291	14	259	358
Fuel Consumed - 3-DOF DR/DV (kg)	279	14	251	335

Figure 4.4: Touchdown statistics [9]

4.4. Low-thrust spacecraft orbital change manoeuvre

Ion thrusters are capable of achieving specific impulses an order of magnitude higher than chemical thrusters, thus resulting in much more efficient propulsion systems ideal for performing orbital change maneuvers. However the thrust force produced by ion thrusters are several orders of magnitude lower. This means that maneuvers using ion thrusters consist of long continues burn and control times which cannot be approximated by instantaneous changes in the velocity as its done with chemical thrusters. This problem is further complicated when including multi-body dynamics, such as while flying through the solar system.

Several researchers have successfully demonstrated the use of RL as a suitable tool for developing

control systems for efficient orbital change maneuvers with low-thrust spacecraft, including interplanetary trajectories [18, 32, 58].

Zavoli and Federici [58]

Miller [32] starts with a Earth-Mars Mass-Optimal transfer with the dynamics being modeled by a two body problem in the sun-centered inertial frame. Each episode was split in 22 time steps in which the agent is given the chance to adjust the controls. The first case used a fixed time step, while the second case used variable time steps controlled by the RL agent.

As a reference the trajectory was also optimized using the Evolutionary Mission Trajectory Generator (EMTG) high-fidelity Parallel Shooting with Finite Burn (PSFB) mode. The initial conditions for the RL simulations where also taken from the EMTG results.

Figures 4.5 end 4.6 show the results for the fixed time-step case. In this case the agent is able to reach the destination, but it ends up consuming more propellant than the reference trajectory. However if the agent is allowed to vary the timestep it is able to prioritize the moments in time in which to make more adjustments, and improve it's efficiency beyond that achieved with EMTG as shown in Figures 4.7 and 4.8.

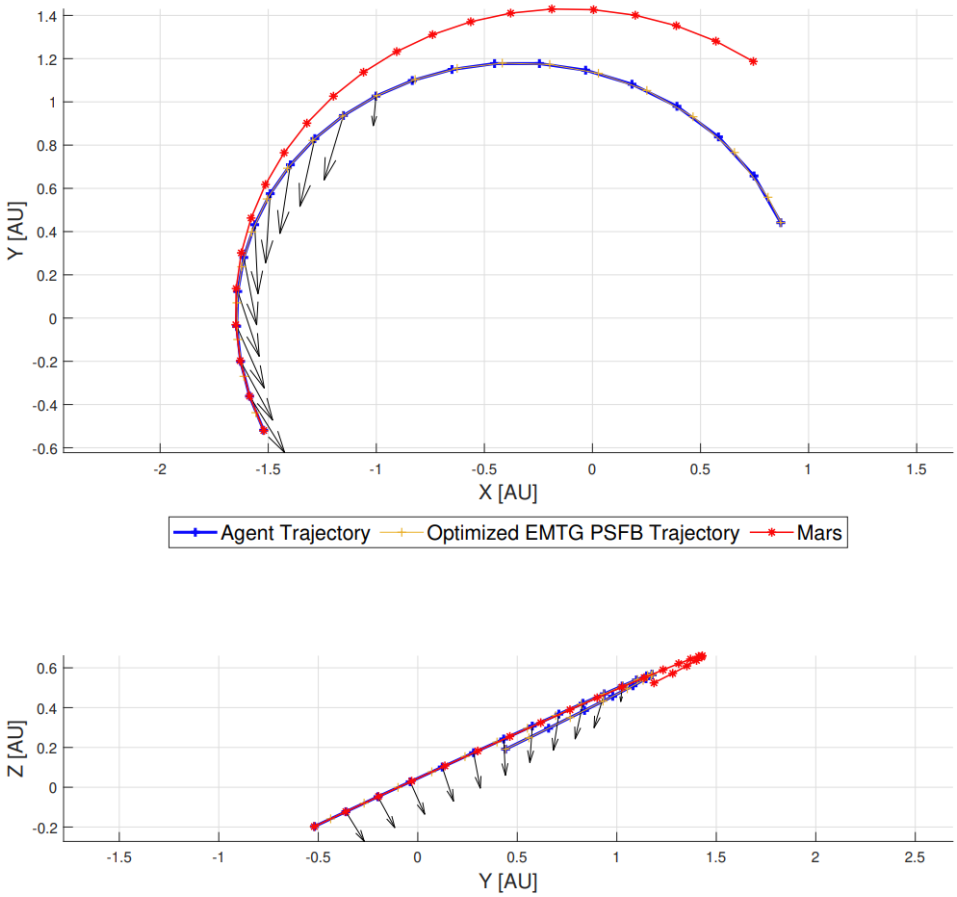


Figure 4.5: Earth-Mars trajectory with fixed timestep [32]

The following experiment by Miller [32] consisted of an agent who's goal was to correct perturbations during transfers between lunar Lyapunov orbits using precalculated reference trajectories. To do this the system was modeled using a planar circular restricted three body problem (CR3BP) and the agent was trained using a reference transfer trajectory from L_1 to L_2 on a path that remained far from the moon.

The performance was then evaluated using new trajectories that the agent had not seen before: an L_1 to L_2 transfer with a close pass of the moon, a L_2 to L_1 remaining far from the moon and a L_2 to L_1

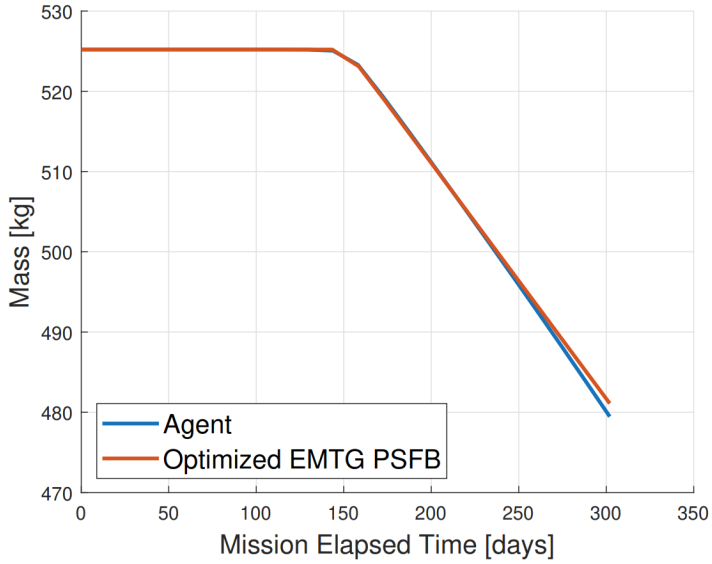


Figure 4.6: Earth-Mars mission mass history with fixed timestep [32]

with a close lunar pass. This was to determine the agent's ability to generalize the solution to many variants of the same problem. The reference trajectories are showed in Figure 4.9.

The error levels were defined to be 10x, 100x and 1000x than what can be reasonably expected according to Guzzetti *et. al.* [**?**]. Table4.1 shows the results of Monte Carlo simulation batches, each consisting of 10000 episodes.

Error Level	Reference 1	Reference 2	Reference 3	Reference 4
10x	99.83%	99.15%	90.61%	99.22%
100x	99.72%	99.07%	89.68%	99.20%
1000x	99.09%	97.49%	87.65%	99.02%

Table 4.1: Monte Carlo Results [32]

The agent performed nearly perfect for simulations following the Reference 1 trajectory, by completing the transfer successfully on more than 99% of episodes. Simulations following the close lunar pass showed a slight decrease in performance, but where also able to complete the task in the vast majority of cases, only having a 97.49% success rate at the largest error level.

The agent was less successful in following the reverse trajectories Reference 3 and 4. The agent applied higher thrust levels compared to the Reference 1 and 2 trajectories, which is a sign that the agent may not have mastered these transfers. However the agent was still consistently completing the transfers successful, showing a nearly perfect success rate for Reference 4 trajectories.

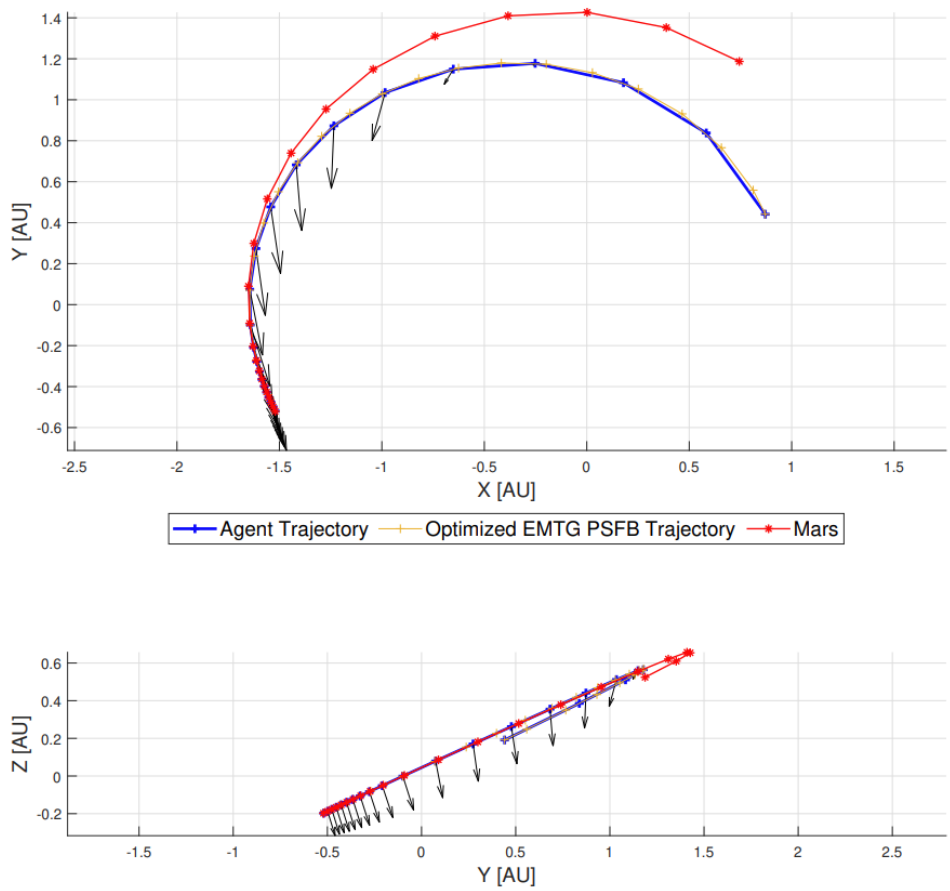


Figure 4.7: Earth-Mars trajectory with variable timestep [32]

Figure 4.8: Earth-Mars mission mass history with variable timestep [32]

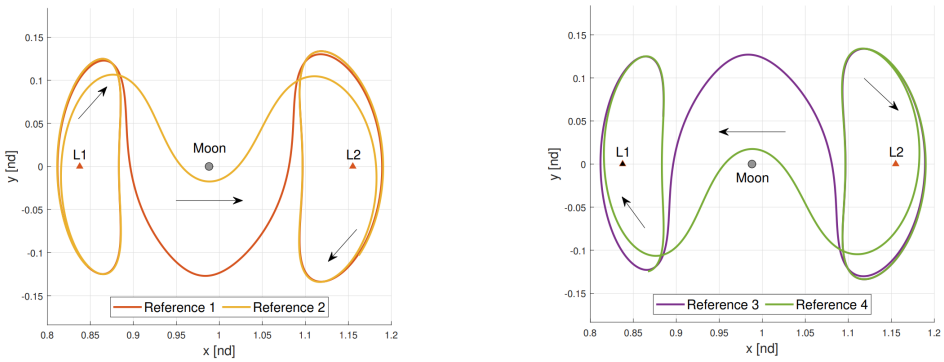


Figure 4.9: Reference trajectories [32]

5 Conclusion

The fist set of research questions focus on the methods commonly used for computer orbital ascent trajectories.

How are orbital ascent trajectories currently computed?

- What are the challenges faced when computing an orbital ascent trajectory?
- What methods are currently used or being researched to solve this problem?
- What are the limitations of methods?

The most common methods used to compute orbital ascent trajectories are based on trajectory optimization, a set of mathematical tools used to compute open-loop solutions to optimal control problems. The reason is because these methods are well suited for solving problems that are non-linear, high dimensional and sensitive to errors, a set of traits which unfortunately true when computing orbital ascent trajectories.

Trajectory optimization methods can be split in two categories, direct and indirect methods. With direct methods the infinite-dimensional trajectory optimization problem is first transcribed to a finite-dimensional approximation which can then be solved using a parameter optimization method. In contrast with indirect methods a surrogate problem is created by analytically deriving the necessary conditions for optimal control and then the new problem is transcribed into a finite-dimensional approximation, which can then be solved using a parameter optimization method.

Indirect methods are more commonly used for computing orbital ascent trajectories, because these tend to be more accurate and provide a better measure of accuracy. However these methods are more complex then direct methods since an analytical model is required and the surrogate problem needs to be derived analytically.

Furthermore during the derivation of the surrogate problem a new set of adjoint variables are added, which also need to be initialized and optimized together the controls and states, and since these adjoint variables don't have a physical representation, it may be difficult to find appropriate initial values when solving the problem.

To ease the complexity of initializing indirect problems, Pontani *et al.* [38–41] proposes the use of Participle Swarm Optimization (PSO) to numerically determine the control, state and adjoint variables after analytically deriving the surrogate problem. Due to its heuristic nature PSO is able to try a wider range of initial conditions and thus significantly lessens the initialization issues while retaining the accuracy that comes with indirect methods.

The main limitation of trajectory optimization methods is that only open-loop solutions can be obtained. These are solutions that consist of an initial condition and a sequence of controls that need to be applied through time. Some drawbacks of this type of solution are a high sensitivity to initial conditions and

lack of guarantee that the solutions converge to the global optimal. Furthermore a stabilizing controller is required to keep the system on the predefined trajectory when applied to real systems.

In contrast closed-loop solutions provide the optimal controls for the entire state space, thus closed-loop solutions are able to guarantee the global optimal solution and can be applied directly to real systems. The drawback is that these solutions are difficult to compute, specially when applied to high dimensional problems.

Originally developed in the 1950's, Dynamic Programming (DP) is a class of methods used to compute closed-loop solutions for optimal control problems. However since these methods require repeated scans of the entire state space, the complexity and computational requirements increases exponentially as more dimensional and states are added to the problem.

These problems can be mitigated or solved by combining DP with various machine learning techniques to produce what is now known as Reinforcement Learning (RL). Thanks to these techniques, RL methods are able to learn closed-loop solutions with only partial views of the state space by generalizing between visited states and actions.

The next research questions focus RL and how it can be used to solve the aforementioned problems.

What is Reinforcement Learning?

- How does RL work?
- What is RL capable of?
- How has RL been used so far?
- What state of the art technological developments make use of RL?

How can Reinforcement Learning methods be used to solve the orbital ascent trajectory optimization problem?

- What advantages do RL methods have over existing methods?
- What properties of RL can be used to circumvent the limitations of current orbital ascent trajectory optimization methods?
- Are RL based orbital ascent trajectory computation methods capable of achieving the accuracy of existing methods?
- How many times should a trained agent be used to make the use of RL cost effective.
- May these RL methods be used to compliment or improve the performance of existing methods?

Both DP and RL are used to solve a class of stochastic optimal control problem known as Markov Decision Process (MDP). These problems consist of two main elements, an agent that makes decisions based on a policy, interacts with its environment and learns. And the environment which consists of everything else. The agent will perform an action which changes the environment and then the environment responds by giving the agent a reward, which it then uses to learn. The goal of the agent is to maximize the cumulative reward it receives through time.

The reward only gives an indication of the immediate effects of the agent's actions, it shows which states are good and bad. These rewards are used to compute the value function. A function that gives an indication of which state-actions pairs are more likely to yield higher rewards in the long run.

Some RL methods take decisions directly from the estimated value functions, however a different class of methods that estimate the policy directly are better able to handle high dimensional problems with continues action spaces. These are known as policy gradient methods. These methods can still take advantage of an estimated value function to speed up the learning process, these methods are known as actor-critic methods, where the policy estimate is the decision taking actor and the value function is the critic.

Various learning techniques exist for RL, the main one being used in this thesis is Proximal Policy Optimization (PPO). The main scheme used in this technique is limiting the difference between two policies during an update within a given trust region. This minimizes the risk of a bad update and

allows multiple updates to the policy to be performed using the same set of samples, thus improving sample efficiency [47, 48].

The use of RL in spacecraft navigation is relatively new and limited to research settings. The main characteristics of RL that of interest are the ability to generalize and autonomously adapt to unknown situations. For example Gaudet and Furfaro [8] successfully explored the use of RL as a control method for hovering in the non-uniform, rotating gravitational field commonly associated with asteroids.

Hovell and Ulrich [12, 13] and Broida and Linares [3] have been developing RL based control system for proximity operation guidance that could be applied in situations where the chaser spacecraft may be required to manoeuvre in the proximity of a potentially uncooperative target object.

Gaudet et al. [9] also demonstrated the use of an RL agent for planetary powered landing, with the aim to develop a landing system capable to navigate within meters from its target. And finally RL has also been successfully demonstrated to be a suitable tool to develop efficient control systems for efficient orbital change manoeuvre with low-thrust spacecraft.

The main project for this thesis is the use of RL to compute closed-loop solutions for orbital ascent trajectories. Although the overall training process is likely more computationally intensive than computing a single solution using trajectory optimization, a already trained agent would be able to compute the optimal or close to optimal solution with just one simulation run.

RL can be a cost effective solution when a large set of optimal trajectories are required, such as during preliminary design of a new launch vehicle or as part of a larger Multidisciplinary Design Optimization (MDO) project. So far most research questions have been answered in this preliminary thesis. The remaining questions that still need to be investigated are:

How can Reinforcement Learning methods be used to solve the orbital ascent trajectory optimization problem?

- Are RL based orbital ascent trajectory computation methods capable of achieving the accuracy of existing methods?
- How many times should a trained agent be used to make the use of RL cost effective.
- May these RL methods be used to compliment or improve the performance of existing methods?

To answer these questions the following plan is proposed. The project will consists out of two main programs, a Trajectory Optimization (TO) based program using the method proposed by Pontani *et al.* [38–41] and program using a novel RL based method.

Since the achieved performance is not known until the terminal state, it is necessary to use a method of supervised or imitation learning to help initialize the models used by the RL agent. The Trajectory Optimization (TO) based program will be used as the supervisor, thus the first step of the project is to develop it.

One of the goals of the project is to check how many times the trained agent will need to be used to make it cost effective over using the Trajectory Optimization (TO) based program for every trajectory. To do this a controlled environnement is needed to run the training and optimizations. In order to create and control these computation environments the code will be developed to run inside Docker containers. These containers can then be deployed into dedicated cloud computing instances for consistent access to computation resources. All logged computation times should then form a reliable measure of computational requirements.

To assess the accuracy of the RL based program, the results of it can be compared to the results yielded by the TO based program. If accurate both should yield trajectories with the same or similar performance, with the performance in this case being measured by propellant utilization.

If the RL based program is not able to achieve the accuracy of the TO based program, then another option is to combine RL methods and TO methods to produce a hybrid program by using the trained RL agent to generate initial conditions for the TO method.

A

Full analytical solution to orbital ascent trajectory optimization

The analytical solution to the orbital ascent trajectory optimization problem as described in *Introduction to space dynamics by William T. Thomson* [52] is shown in this section.

A.1. Single stage performance

A rocket is a variable mass vehicle which produces thrust by ejecting high speed particles. Neglecting atmospheric forces and assuming a uniform gravitational field during vertical flight results in the following equation for the vehicle's acceleration $\frac{dv}{dt}$, where T is the thrust, m the mass and g_0 the gravitational acceleration at the earth's surface.

$$\frac{dv}{dt} = \frac{T}{m} - g_0 \tag{A.1}$$

Since the rocket is constantly losing mass, the mass flow $\frac{dm}{dt}$ is negative and thus the thrust can be written as shown below if a term due to the difference in exhaust and ambient pressure is neglected.

$$T = -u \cdot \frac{dm}{dt} \tag{A.2}$$

Here u is the velocity of the expelled mass, also known as the exhaust velocity. The specific impulse I_{sp} is a measure of the performance of a rocket engine and it's relationship to the exhaust velocity as taken from the book [52] is as follows,

$$u = g_0 I_{sp} \tag{A.3}$$

Substituting Equation (A.2) into Equation (A.1) and integrating yields the following equation for the velocity v.

$$v - v_0 = u \ln \frac{m_0}{m} - g_0 t \tag{A.4}$$

Where m is the mass at any time t. Substituting the burnout time t_{b0} and burnout mass m_{b0} into the equation results in an expression for the maximum attainable velocity for vertical flight.

$$v_{b0} - v_0 = u \ln \frac{m_0}{m_{b0}} - g_0 t_{b0} \tag{A.5}$$

The thrust parameter $\mathcal{R}$ is now introduced. This is the initial thrust over mass ratio and its relationship to the initial acceleration a_0 is as follows,

$$\mathcal{R} = \frac{T}{m_0} = \frac{a_0}{g_0} + 1 \tag{A.6}$$

The duration for powered flight is then,

$$t_{b0} = g_0 I_{sp} \left(\frac{m_0 - m_{b0}}{T} \right) = \frac{I_{sp}}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) \tag{A.7}$$

Using Equations (A.3) and (A.7), Equation (A.5) can now be written as,

$$v_{b0} - v_0 = g_0 I_{sp} \left[\ln \frac{m_0}{m_{b0}} - \frac{1}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) \right] \tag{A.8}$$

It is possible to determine the distance traveled during powered flight by integrating Equation (A.4). A realistic assumption is that the mass during flight is known and that a constant rate of fuel mass leads to constant thrust. Thus the following can be defined,

$$dt = \frac{dm}{\dot{m}} \qquad \text{and} \qquad -\dot{m} = \frac{m_0 - m_{b0}}{t_{b0}} = \text{constant} \tag{A.9}$$

so that

$$\int_0^{t_{b0}} (\ln m)\, dt = \frac{1}{\dot{m}} \int_{m_0}^{m_{b0}} (\ln m)\, dm = \frac{m}{\dot{m}} (\ln m - 1) \Big|_{m_0}^{m_{b0}} \tag{A.10}$$

$$= \frac{m_0 - m_{b0}}{\dot{m}} - \frac{m_0}{\dot{m}} \ln m_0 + \frac{m_{b0}}{\dot{m}} \ln m_{b0} \tag{A.11}$$

The distance traveled then becomes,

$$h_{b0} = u t_{b0} \left[1 - \frac{1}{(m_0/m_{b0} - 1)} \ln \frac{m_0}{m_{b0}} \right] + v_0 t_{b0} - \frac{1}{2} g_0 t_{b0}^2 \tag{A.12}$$

$$= g_0 \frac{I_{sp}^2}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) \left[1 - \frac{1}{(m_0/m_{b0} - 1)} \ln \frac{m_0}{m_{b0}} \right] + v_0 \frac{I_{sp}}{\mathcal{R}} \left(1 - \frac{m_{b0}}{m_0} \right) - \frac{1}{2} g_0 \frac{I_{sp}^2}{\mathcal{R}^2} \left(1 - \frac{m_{b0}}{m_0} \right)^2 \tag{A.13}$$

After burnout the rocket continues coasting with the velocity only changing due to the gravitational acceleration. Unfortunately due to the altitude in this phase of flight it is not possible to assume a uniform gravitational field.

Since the system does not lose or gain energy while coasting it is possible to get the distance traveled by equating the kinetic energy at burnout with the work done by the *gravitational force* $m_{b0} g_0 \left(\frac{R}{r} \right)^2$. Thus the equation for coasting distance becomes,

$$h_c = \frac{v_{b0^2}}{2 g_0} \frac{(R + h_{b0})^2}{R^2 - (v_{b0}^2 / 2 g_0)(R + h_{b0})} \tag{A.14}$$

where $r_{b0} = R + h_{b0}$ has been substituted. The total height can then be calculated by summing Equations (A.12) and (A.14).

Equations (A.8) and (A.14) indicate that the performance of a single stage rocket depends on the specific impulse I_{sp} and thrust ratio $\mathcal{R}$ and the mass ratio $\mu = \frac{m_0}{m_{b0}}$. The effect of varying these quantities on the burnout velocity or height can be found by considering Equations (A.8) and (A.12) to be in the form,

$$v_{b0} = f_1\left(I_{sp}, \mathcal{R}, \mu\right) \tag{A.15}$$

$$h_{b0} = f_2\left(I_{sp}, \mathcal{R}, \mu\right) \tag{A.16}$$

The change in burnout velocity is then defined by the following equation,

$$dv_{b0} = \frac{\delta f_1}{\delta I_{sp}} dI_{sp} + \frac{\delta f_1}{\delta \mathcal{R}} d\mathcal{R}_{sp} + \frac{\delta f_1}{\delta \mu} d\mu \tag{A.17}$$

The optimum burnout velocity will be at $dv_{b0} = 0$, this defines the constraint on the three quantities.

A.2. Optimization of multistage rockets

It can be shown using the equations from the previous section that with achievable mass ratios, thrust ratios and specific impulse, it is not possible to reach earth orbital velocities using single stage rockets. Thus multistage launchers are required for space travel.

In a multistage rocket the burnout velocity of the previous stage is the initial velocity of the next. The maximum velocity by multistage rockets can then be computed as the sum of the single stage velocities given by Equation (A.5). Ignoring gravity loss due to burn-time results in the following equation for the maximum velocity of an N-stage rocket,

$$v_m = \sum_{i=1}^{N} u_i \ln \mu_i \tag{A.18}$$

Assuming this velocity v_m is specified, the mass ratios for each stage must be chosen such that the overall mass ratio $\frac{m_{01}}{P}$ is minimized, where m_{01} is the take-off mass and P the payload mass. To do this it is necessary to express the overall mass ratio in terms of the individual mass ratios μ_i.

Since the empty structure of the stages are to be discarded, the initial mass of each new stage is equal to the initial mass of the previous stage minus the propellant burned and the empty structure that is thrown off. Thus the overall mass ratio may be written as follows.

$$\frac{m_{01}}{P} = \frac{m_{01}}{m_{01} - m_{p1} - m_{s1}} \frac{m_{02}}{m_{02} - m_{p2} - m_{s2}} \dots \frac{m_{0N}}{P} \tag{A.19}$$

The structural factor β_i is now introduced.

$$\beta_i = \frac{m_{si}}{m_{pi} + m_{si}} \tag{A.20}$$

Equation (A.19) can then be rewritten into,

$$\frac{m_{01}}{P} = \frac{\mu_1\left(1-\beta_1\right)}{1-\mu_1\beta_1} \frac{\mu_2\left(1-\beta_2\right)}{1-\mu_2\beta_2} \dots \frac{\mu_N\left(1-\beta_N\right)}{1-\mu_N\beta_N} \tag{A.21}$$

If $\frac{m_{01}}{P}$ needs to be minimized, then the same result can be achieved my minimizing $\ln \frac{m01}{P}$ as well.

$$\ln \frac{m_{01}}{P} = \sum_{i=1}^{N} \ln \frac{\mu_i\left(1-\beta_i\right)}{1-\mu_i\beta_i} = \sum_{i=1}^{N} \left[\ln \mu_i + \ln\left(1-\beta_i\right) - \ln\left(1-\mu_i\beta_i\right)\right] \tag{A.22}$$

This equation does not contain the constraint imposed by the specified velocity in Equation A.18. It can be added using the Lagrange multiplier method which results in the following,

$$\ln \frac{m_{01}}{P} = \sum_{i=1}^{N} \left[\ln \mu_i + \ln (1-\beta_i) - \ln (1-\mu_i \beta_i) + \lambda \left\{u_i \ln \mu_i - v_m\right\}\right] \tag{A.23}$$

In order to solve for the optimal mass ratios μ_i, Equation (A.23) must be differentiated with respect to μ_i. Performing this differentiation and rewriting results in N equations in the form of,

$$\mu_i = \frac{1+\lambda u_i}{\lambda u_i \beta_i} \tag{A.24}$$

Substituting this into Equation (A.18) yields the following expression which, assuming u_i, β_i and v_m are known, may be used to solve for λ.

$$v_m = \sum_{i=1}^{N} u_i \ln \frac{1+\lambda u_i}{\lambda u_i \beta_i} \tag{A.25}$$

With λ known the mass ratios μ_i may be calculated using Equation (A.24).

A.3. Optimal thrust attitude angle policy

To simplify this problem the rocket is assumed to be rigid and the aerodynamics are neglected. In general the length of the powered flight path is short compared to the earth's radius, thus it's safe to replace the central gravitational field by a constant-plane parallel-force field.

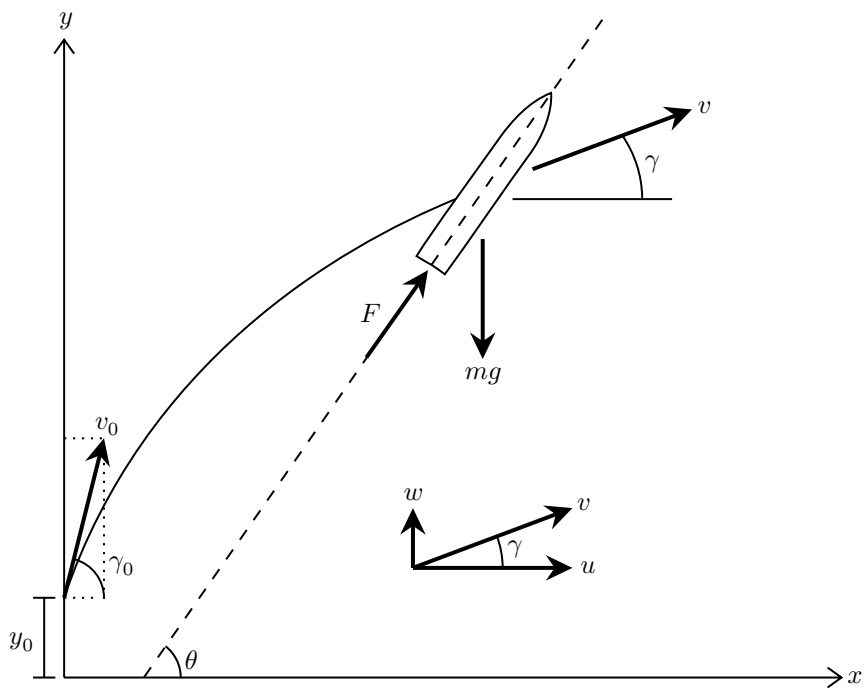


Figure A.1: Powered flight trajectory[52].

The geometry of the problem is shown in Figure A.1. Where θ is the thrust attitude angle and γ the flight path angle. Gravity acts along to the negative y axis. Letting u and w be the x and y component

of the velocity v results in the following equations of motion.

$$\dot{u} = \frac{F}{m}\cos\theta \tag{A.26}$$

$$\dot{w} = \frac{F}{m}\sin\theta - g \tag{A.27}$$

$$\dot{y} = w \tag{A.28}$$

$\frac{F}{m}$ and θ vary with time and have a significant effect on the trajectory. Assuming that $\frac{F}{m}$ is known then the problem can be defined as finding the $\theta(t)$ that results the maximum horizontal velocity U at a specified altitude Y. The time T corresponds to the instant in time where $y = Y$. Since the timing of this event depends on $\theta(t)$, T is unknown. However it is assumed that all propellant is burned before $t = T$.

The problem can thus be defined as maximizing the time integral of Equation (A.26) shown below, with constraints.

$$U = u_0 + \int_0^T \frac{F}{m}\cos\theta\, dt \tag{A.29}$$

The goal is to reach this maximum at a specified value of y under the condition that $w_{t=T} = 0$. So Equations (A.27) and (A.28) are the constraints that limit the allowable variations of Equation (A.29). Using the Lagrange multiplier method to add these constraints results in the following equation.

$$U = u_0 + \int_0^T \left[\frac{F}{m}\cos\theta + \lambda_1\left(\dot{w} - \frac{F}{m}\sin\theta + g\right) + \lambda_2\left(\dot{y} - w\right)\right] dt \tag{A.30}$$

With the boundary conditions being,

At $t = 0$

$$y = y_0 \qquad x = 0$$
$$u_0 = v_0\cos\gamma_0 \qquad w_0 = w_0\sin\gamma_0$$

At $t = T$

$$y = Y \qquad w = 0$$

The initial velocity v_0 is considered fixed, but the initial flight path angle γ_0 is left undetermined.

This optimization is solved using variational calculus. Thus the next step is to get an equation for δU, the variation of Equation (A.30). However determining the variation of an integral is not trivial. According to derivation in appendix C of the book [52], the variation of the following integral is shown in Equation (A.32).

$$I = \int_a^b f\left(t, z, \frac{dz}{dt}\right) \tag{A.31}$$

$$\delta I = \frac{\delta f}{\delta z'}\delta z\Big|_a^b + \int_a^b \left(\frac{\delta f}{\delta z} - \frac{d}{dt}\frac{\delta f}{\delta z'}\right)\delta z\, dt \tag{A.32}$$

Applying Equation (A.32) to Equation (A.30) results in the following equation for the variation of U.

$$\begin{aligned}\delta U = &- v_0\sin\gamma_0\,\delta\gamma_0 + \lambda_1\delta w\Big|_0^T + \lambda_2\delta y\Big|_0^T \\ &- \int_0^T \left[\frac{F}{m}\left(\sin\theta + \lambda_1\cos\theta\right)\delta\theta + \frac{d\lambda_2}{dt}\delta y + \left(\frac{d\lambda_1}{dt} + \lambda_2\right)\delta w\right] dt = 0\end{aligned} \tag{A.33}$$

Since the value of y is known at $t = 0$ and $t = T$, it's variation at the endpoints must be zero, thus the third term can be eliminated. The variation δw at $t = 0$ is,

$$\delta w\Big|_0 = \delta(v_0 \sin\gamma_0) = v_0 \cos\gamma_0 \, \delta\gamma_0 \tag{A.34}$$

At $t = T$, w can be calculated by integrating Equation (A.27) into the equation shown below.

$$w = \int_0^T \frac{F}{m} \sin\theta dt - gT + v_0 \sin\gamma_0 \tag{A.35}$$

Thus it's variation at $t = T$ is,

$$\delta w\Big|^T = \int_0^T \frac{F}{m} \cos\theta \, \delta\theta \, dt + v_0 \, \delta\gamma_0 \sin\gamma_0 \tag{A.36}$$

For the optimal curve w becomes zero at $t = T$, however for the varied curve it becomes zero at a different time $t = T + \delta T$. Since $\frac{F}{m}$ is not varied for $t > T$ the expression for $\delta w|^T$ must equal $g\delta T$. Substituting these equations into Equation (A.33) yields,

$$\begin{aligned} \delta U = &- v_0(\sin\gamma_0 + \lambda_{1,t=0}\cos\gamma_0)\,\delta\gamma_0 + g\lambda_{1,t=T}\,\delta T \\ &- \int_0^T \left[\frac{F}{m}(\sin\theta + \lambda_1\cos\theta)\,\delta\theta + \frac{d\lambda_2}{dt}\delta y + \left(\frac{d\lambda_1}{dt} + \lambda_2\right)\delta w\right] dt = 0 \end{aligned} \tag{A.37}$$

Since the rest of the variations are arbitrary, δU can only be zero if

$$\tan\gamma_0 = -\lambda_{1,t=0} \tag{A.38}$$

$$\lambda_{1,t=T} = 0 \tag{A.39}$$

$$\tan\theta = -\lambda_1 \tag{A.40}$$

$$\frac{d\lambda_2}{dt} = 0 \tag{A.41}$$

$$\frac{d\lambda_1}{dt} + \lambda_2 = 0 \tag{A.42}$$

The last two equations give,

$$\lambda_2 = -C_2 \qquad \text{a constant} \tag{A.43}$$

$$\frac{d\lambda_1}{dt} - C_2 = 0 \tag{A.44}$$

$$\lambda_1 = C_1 + C_2 t \tag{A.45}$$

Substituting λ_1 for $t = 0$ and $t = T$ into Equations (A.38) and (A.39), results in expressions for constants C_1 and C_2.

$$C_1 = -\tan\gamma_0 \tag{A.46}$$

$$C_2 = \frac{1}{T}\tan\gamma_0 \tag{A.47}$$

Combining Equations (A.40, A.45, A.46, A.47) yields an expression for the thrust attitude angle θ.

$$\tan\theta = \left(1 - \frac{t}{T}\right)\tan\gamma_0 \tag{A.48}$$

Equation (A.48) gives an expression for the thrust attitude angle θ if the initial flight path angle γ_0 is allowed to variate. If γ_0 is given a fixed quantity, a similar procedure starting from Equation (A.33) can be followed with the added condition that $\delta\gamma_0 = 0$. This results in the following.

$$\tan\theta = \tan\theta_0 - C_2 t \tag{A.49}$$

A.4. Optimal propellant utilization policy

The previous section resulted in the optimal thrust attitude angle θ policy that results in a maximum horizontal velocity U for a specified thrust ratio $\frac{F}{m}$ policy. However the maximum attainable U still depends on what policy is chosen for $\frac{F}{m}$ or in other words, the manner of propellant utilization.

It is important to note that there is a maximum attainable velocity V_k. From Equation (A.5) it is clear that the burn-time has a negative effect on V_k, thus the optimal function for $V(t)$ must be discontinues, in other words, the propellant must be burned instantly. By neglecting all forces except thrust, V_k is only a function of the specific impulse and mass ratio.

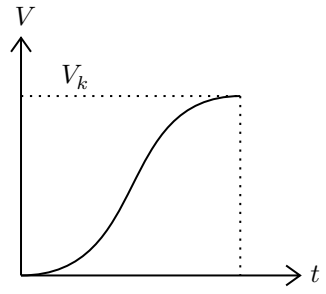


Figure A.2: Possible variation of V

$$V_k = gI_{sp} \ln \frac{m_0}{m_{b0}} \tag{A.50}$$

Assuming the specific impulse is independent of the propellant utilization policy, it is convenient to make the following substitutions.

$$\frac{dV}{dt} = \frac{F}{m} \tag{A.51}$$

$$V = \int_0^t \frac{F}{m} dt \tag{A.52}$$

Here is V the rocket thrust velocity with the maximum attainable value being V_k. V is only a function of time and depends on the manner in which fuel is consumed. Both F and m are positive values thus the slope for V is bounded between 0 and infinity. Figure A.2 shows a possible variation of V.

Assuming the initial values v_0 and γ_0 are fixed and that the policy for the thrust attitude angle in Equation (A.49) is followed. Then Equation (A.30) may be rewritten into,

$$U = \int_0^T \left[(\cos\theta - \lambda_1 \sin\theta) \frac{dV}{dt} + \lambda_1 (\dot{w} + g) + \lambda_2 (\dot{y} - w) \right] dt \tag{A.53}$$

Since the optimal thrust attitude policy is followed, then all variations have already been handled in the previous section, except for δV. Thus only the first term of the integral needs to be considered.

In order to apply Equation (A.32) to Equation (A.53) the following is needed,

$$\frac{\delta f}{\delta V} = 0 \qquad \frac{\delta f}{\delta \dot{V}} = (\cos\theta - \lambda_1 \sin\theta) \tag{A.54}$$

Since the values for V at the start and end time are fixed, it's variation δV at $t = 0$ and $t = T$ are zero. Thus the variation of Equation (A.53) is as follows,

$$\begin{aligned} \delta U &= -\int_0^t \frac{d}{dt} (\cos\theta - \lambda_1 \sin\theta) \delta V dt \\ &= \int_0^t \left[(\sin\theta + \lambda_1 \cos\theta) \dot{\theta} + \sin\theta \frac{d\lambda_1}{dt} \right] \delta V dt \end{aligned} \tag{A.55}$$

From Equations (A.55) and (A.49) it's possible to get,

$$\lambda_1 = -\tan\theta = -\tan\theta_0 + C_2 t \tag{A.56}$$

$$\frac{\lambda_1}{dt} = -\frac{\dot{\theta}}{\cos^2\theta} = C_2 \tag{A.57}$$

Using these expressions, δU may be written as,

$$\delta U = C_2 \int_0^T (\sin\theta)\,\delta V dt \tag{A.58}$$

$$= C_2 \int_0^T \frac{\tan\theta_0 - C_2 t}{\sqrt{1 + (\tan\theta_0 - C_2 t)^2}} \delta V dt \tag{A.59}$$

$$= C_2 \int_0^T \Phi\ \delta V dt = 0 \tag{A.60}$$

Where,

$$\Phi = \frac{\tan\theta_0 - C_2 t}{\sqrt{1 + (\tan\theta_0 - C_2 t)^2}} \tag{A.61}$$

Before continuing some details are reiterated.

- Since U is being maximized, any variation δU must be negative.
- The velocity V may not exceed V_k.
- Since the slope of V is bounded between zero and infinity, it is not possible for V to be negative.
- The optimal function for $V(t)$ is discreet, meaning only instantaneous burns.

It is possible for Φ to be always positive, always negative or vary from positive to negative with time.

1. If Φ is always positive, then any variation δV must be negative in order for δU to be negative. Since $V(t)$ must always be larger than zero, then all the propellant must be burned instantaneously at $t = 0$ so that any variation δV from the optimal $V(t)$ is negative.
2. If Φ is always negative, then any variation δV must be positive in order for δU to be negative. Since $V(t)$ must always be lower than V_k, then all the propellant must be burned instantaneously at $t = T$.
3. If Φ varies from positive to negative. Then part of the propellant must be instantaneously burned at $t = 0$, and the rest and $t = T$.

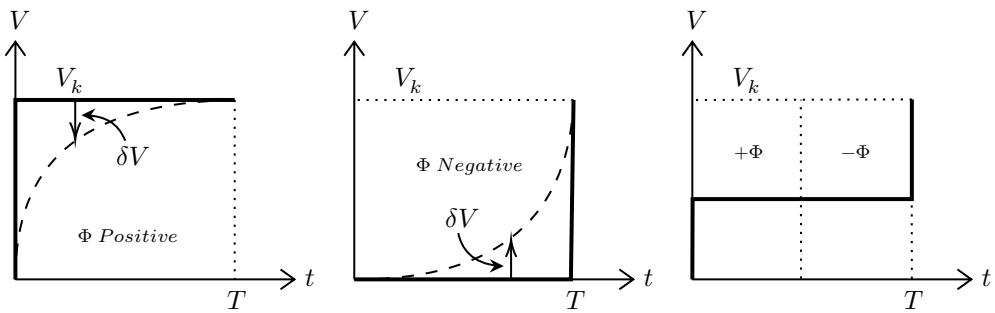


Figure A.3: Optimal $V(t)$ for different options for Φ[52].

A.5. Gravity turn

It was previously shown that the optimal thrust attitude angle policy is given by the following equation,

$$\tan\theta = \left(1 - \frac{t}{T}\right)\tan\gamma_0 \tag{A.62}$$

One drawback of this policy is that it results in large angles of attack $\alpha = \theta - \gamma$. This is not an issue when flying outside of the atmosphere, but when flying within the atmosphere it results in increased aerodynamic forces, thus increasing aerodynamic losses and risk of structural damage. Thus for flights within the atmosphere a trajectory known as the *gravity turn* is followed. In a gravity turn the thrust vector is kept parallel to the velocity vector at all times starting from a non-vertical initial velocity v_0. This is a zero angle of attack or zero lift trajectory.

For convenience the flight path angle ψ is now measured from the vertical. By neglecting aerodynamic forces and assuming a uniform gravitational field, it is possible to write the equation for the forces along the tangential and normal directions as follows,

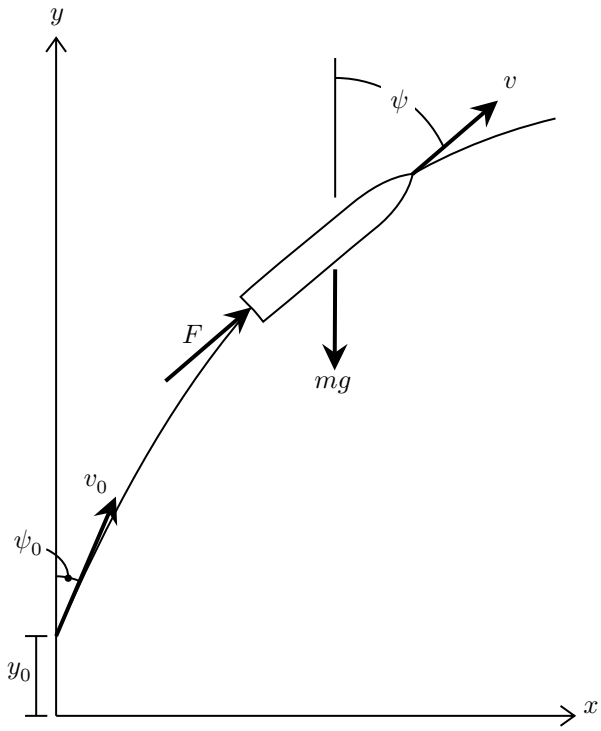


Figure A.4: Gravity turn[52].

$$\frac{1}{g}\frac{dv}{dg} = \frac{F}{mg} - \cos\psi \tag{A.63}$$

$$\frac{v}{g}\frac{d\psi}{dt} = \sin\psi \tag{A.64}$$

These equations are non-linear and there is no analytical solutions when $\frac{F}{mg}$ varies, thus $\frac{F}{mg}$ needs to be constant in order to solve the gravity turn trajectory. Since $m = m_0 - \dot{m}t$, this means that the thrust F must decrease with time as well.

However in reality it is reasonable to assume that $F = constant$. Thus for a realistic solutions it's not possible to assume that $\frac{F}{mg}$ doesn't vary. One way solve it, is by assuming $\frac{F}{mg}$ is constant for a short amount of time and perform a step-by-step numerical integration using the analytical solution for constant $\frac{F}{mg}$ on each interval.

A new parameter z is introduced and defined as,

$$z = \tan\frac{1}{2}\psi = \sqrt{\frac{1-\cos\psi}{1+\cos\psi}} = \frac{\sin\psi}{1+\cos\psi} \tag{A.65}$$

Then

$$z^2 = \frac{1-\cos\psi}{1+\cos\psi} \tag{A.66}$$

and

$$\frac{1-z^2}{1+z^2} = \cos\psi \tag{A.67}$$

Differentiating Equation (A.65),

$$\frac{dz}{dt} = \frac{1}{2\cos^2\left(\frac{\psi}{2}\right)}\frac{d\psi}{dt} \tag{A.68}$$

Thus,

$$\frac{d\psi}{dt} = (1 + \cos\psi)\frac{dz}{dt} \tag{A.69}$$

By defining $n = \frac{F}{mg}$ and substituting the previous results into Equations (A.63) and (A.64) results in,

$$\frac{1}{g}\frac{dv}{dt} = n - \frac{1 - z^2}{1 + z^2} \tag{A.70}$$

$$\frac{v}{g}\frac{dz}{dt} = z \tag{A.71}$$

Eliminating dt between the two equations yields,

$$\frac{dv}{v} = n\frac{dz}{z} - \frac{1 - z^2}{1 + z^2}\frac{dz}{z} \tag{A.72}$$

Integrating,

$$\ln v = \ln z^n = \ln\frac{1 + z^2}{z} + \ln C' \tag{A.73}$$

or

$$v = Cz^{n-1}\left(1 + z^2\right) \tag{A.74}$$

The constant C can be calculated from the initial values with,

$$C = \frac{v_0}{z_0^{n-1}\left(1 + z_0^2\right)} \tag{A.75}$$

Substituting Equation (A.74) into (A.71) and integrating,

$$\begin{aligned} t &= \frac{C}{g}\int_{z_0}^{z} z^{n-2}\left(1 + z^2\right)dz \\ &= \frac{C}{g}z^{n-2}\left(\frac{1}{n-1} + \frac{z^2}{n+1}\right)\Bigg|_{z_0}^{z} \end{aligned} \tag{A.76}$$

Equation (A.76) gives t as a function of z and any initial condition C. Thus ψ is also known for any C and t. Equation (A.74) gives v in terms of C, t and z. So Equations (A.65), (A.75) and (A.76) give the analytical solution for the gravity turn with a constant $\frac{F}{mg}$.

In order to solve the gravity turn with varying $\frac{F}{mg}$ Equation (A.65) can be used to calculate values for z, Equation (A.75) to calculate C, define $n = \frac{F}{mg}$ at $t = 0$ and v can be calculated using Equation (A.74). Then a value of ψ slightly higher than ψ_0 must chosen and Δt can then be calculated from Equation (A.76). The increment in displacement is then,

$$\Delta x = \frac{1}{2}\left(v_0\sin\psi_0 + v\sin\psi\right)\Delta t \tag{A.77}$$

$$\Delta y = \frac{1}{2}\left(v_0\cos\psi_0 + v\cos\psi\right)\Delta t \tag{A.78}$$

This procedure can then be repeated with the values at the new point to obtain the full trajectory.

Bibliography

[1] Yoshua Bengio. *Learning deep architectures for AI*. Now Publishers Inc, 2009.

[2] John T Betts. *Practical methods for optimal control and estimation using nonlinear programming*. SIAM, 2010.

[3] Jacob Broida and Richard Linares. Spacecraft rendezvous guidance in cluttered environments via reinforcement learning. In *29th AAS/AIAA Space Flight Mechanics Meeting*, pages 1–15, 2019.

[4] Anthony Calise, Sunil Tandon, Daniel Young, and Sungwan Kim. Further improvements to a hybrid method for launch vehicle ascent trajectory optimization. In *18th applied aerodynamics conference*, page 4261, 2000.

[5] Victoria Coverstone-Carroll and Steven N Williams. Optimal low thrust trajectories using differential inclusion concepts. 1994.

[6] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[7] Peter F Gath and Anthony J Calise. Optimization of launch vehicle ascent trajectories with path constraints and coast arcs. *Journal of Guidance, Control, and Dynamics*, 24(2):296–304, 2001.

[8] Brian Gaudet and Roberto Furfaro. Robust spacecraft hovering near small bodies in environments with unknown dynamics using reinforcement learning. In *AIAA/AAS Astrodynamics Specialist Conference*, page 5072, 2012.

[9] Brian Gaudet, Richard Linares, and Roberto Furfaro. Deep reinforcement learning for six degree-of-freedom planetary powered descent and landing. *arXiv preprint arXiv:1810.08719*, 2018.

[10] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, et al. Deep q-learning from demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

[12] Kirk Hovell and Steve Ulrich. On deep reinforcement learning for spacecraft guidance. In *AIAA Scitech 2020 Forum*, page 1600, 2020.

[13] Kirk Hovell and Steve Ulrich. Deep reinforcement learning for spacecraft proximity operations guidance. *Journal of Spacecraft and Rockets*, 58(2):254–264, 2021.

[14] Reza Jamilnia and Abolghasem Naghash. Simultaneous optimization of staging and trajectory of launch vehicles using two different approaches. *Aerospace Science and Technology*, 23(1): 85–92, 2012.

[15] Jaime L Junell, Erik-Jan Van Kampen, Coen C de Visser, and Q Ping Chu. Reinforcement learning applied to a quadrotor guidance law in autonomous flight. In *AIAA Guidance, Navigation, and Control Conference*, page 1990, 2015.

[16] Matthew Kelly. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4):849–904, 2017.

[17] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. URL `https://arxiv.org/abs/1412.6980`.

[18] Daniel S Kolosa. A reinforcement learning approach to spacecraft trajectory optimization. 2019.

[19] Eric Liang, Richard Liaw, Philipp Moritz, Robert Nishihara, Roy Fox, Ken Goldberg, Joseph E Gonzalez, Michael I Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. arxiv e-prints, page. *arXiv preprint arXiv:1712.09381*, 2017.

[20] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[21] A Miele and S Mancuso. Optimal ascent trajectories and feasibility of next-generation orbital spacecraft. *Journal of Optimization Theory and Applications*, 97(3):519–550, 1998.

[22] A Miele and S Mancuso. On feasibility of suborbital and orbital rocket-powered spacecraft. *Memorie della Accademia delle Scienze di Torino*, 24:83–109, 2000.

[23] A Miele and S Mancuso. Optimal trajectories for earth–moon–earth flight. *Acta Astronautica*, 49 (2):59–71, 2001.

[24] A Miele and T Wang. Nominal trajectories for the aeroassisted flight experiment. *Journal of the Astronautical Sciences*, 41(2):139–163, 1993.

[25] A Miele and T Wang. Optimal transfers from an earth orbit to a mars orbit. *Acta Astronautica*, 45 (3):119–133, 1999.

[26] A Miele and T Wang. Multiple-subarc gradient-restoration algorithm, part 1: algorithm structure. *Journal of Optimization Theory and Applications*, 116(1):1–17, 2003.

[27] A Miele and T Wang. Multiple-subarc gradient-restoration algorithm, part 2: application to a multistage launch vehicle design. *Journal of Optimization Theory and Applications*, 116(1):19–39, 2003.

[28] A Miele, R Eo Pritchard, and JN Damoulakis. Sequential gradient-restoration algorithm for optimal control problems. *Journal of Optimization Theory and Applications*, 5(4):235–282, 1970.

[29] A Miele, T Wang, and WW Melvin. Optimal take-off trajectories in the presence of windshear. *Journal of optimization theory and applications*, 49(1):1–45, 1986.

[30] A Miele, T Wang, and S Mancuso. Assessment of launch vehicle advances to enable human mars excursions. *Acta Astronautica*, 49(11):563–580, 2001.

[31] A Miele, MW Weeks, and M Ciarcia. Optimal trajectories for spacecraft rendezvous. *Journal of optimization theory and applications*, 132(3):353–376, 2007.

[32] Daniel Daniel Martin Miller. *Low-thrust Spacecraft guidance and control using proximal policy optimization*. PhD thesis, Massachusetts Institute of Technology, 2020.

[33] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.

[34] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937, 2016.

[35] Karthik Narasimhan, Tejas Kulkarni, and Regina Barzilay. Language understanding for text-based games using deep reinforcement learning. *arXiv preprint arXiv:1506.08941*, 2015.

[36] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Icml*, volume 99, pages 278–287, 1999.

[37] Binfeng Pan, Ping Lu, Xun Pan, and Yangyang Ma. Double-homotopy method for solving optimal control problems. *Journal of Guidance, Control, and Dynamics*, 39(8):1706–1720, 2016.

[38] Mauro Pontani. Particle swarm optimization of ascent trajectories of multistage launch vehicles. *Acta Astronautica*, 94(2):852–864, 2014. ISSN 0094-5765. doi: https://doi.org/10.1016/j.actaastro.2013.09.013. URL https://www.sciencedirect.com/science/article/pii/S0094576513003676.

[39] Mauro Pontani and Giampaolo Cecchetti. Ascent trajectories of multistage launch vehicles: Numerical optimization with second-order conditions verification. *International Scholarly Research Notices*, 2013, 2013.

[40] Mauro Pontani and Bruce Conway. Optimal low-thrust orbital maneuvers via indirect swarming method. *Journal of Optimization Theory and Applications*, 162(1):272–292, 2014.

[41] Mauro Pontani and Paolo Teofilatto. Simple method for performance evaluation of multistage rockets. *Acta Astronautica*, 94(1):434–445, 2014.

[42] Jette Randløv and Preben Alstrøm. Learning to drive a bicycle using reinforcement learning and shaping. In *ICML*, volume 98, pages 463–471. Citeseer, 1998.

[43] Anil V Rao. A survey of numerical methods for optimal control. *Advances in the Astronautical Sciences*, 135(1):497–528, 2009.

[44] Brian H Rishikof, Bernell R McCormick, Robert E Pritchard, and Steven J Sponaugle. Segram: A practical and versatile tool for spacecraft trajectory optimization. *Acta Astronautica*, 26(8-10): 599–609, 1992.

[45] Woongrae Roh and Youdan Kim. Trajectory optimization for a multi-stage launch vehicle using time finite element and direct collocation methods. *Engineering optimization*, 34(1):15–32, 2002.

[46] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

[47] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015. URL https://arxiv.org/abs/1502.05477.

[48] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[49] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. PMLR, 2014.

[50] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[51] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[52] William Tyrrell Thomson. *Introduction to space dynamics*. Courier Corporation, 2012.

[53] Oskar Von Stryk and Roland Bulirsch. Direct and indirect methods for trajectory optimization. *Annals of operations research*, 37(1):357–373, 1992.

[54] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3-4):279–292, 1992.

[55] Eric Wiewiora. Reward shaping., 2010.

[56] Stefan Willis, Dario Izzo, and Daniel Hennes. Reinforcement learning for spacecraft maneuvering near small bodies. *American Institute of Aeronautics and Astronautics, Napa*, pages 16–277, 2016.

[57] David H Wolpert, Kevin R Wheeler, and Kagan Tumer. Collective intelligence for control of distributed dynamical systems. *EPL (Europhysics Letters)*, 49(6):708, 2000.

[58] Alessandro Zavoli and Lorenzo Federici. Reinforcement learning for low-thrust trajectory design of interplanetary missions. *arXiv preprint arXiv:2008.08501*, 2020.

[59] Yaru Zheng, Xiaoyu Fu, Ming Xu, Qinglong Li, and Mingpei Lin. Ascent trajectory design of small-lift launch vehicle using hierarchical optimization. *Aerospace Science and Technology*, 107: 106285, 2020.